
Generalised Linear Models in Deep Bayesian RL with Learnable Basis Functions

Jingyang You and Hanna Kurniawati
School of Computing
Australian National University
{jingyang.you, hanna.kurniawati}@anu.edu.au

Abstract

Bayesian Reinforcement Learning (BRL), a subclass of Meta-Reinforcement Learning (Meta-RL), provides a principled framework for generalisation by explicitly incorporating Bayesian task parameters into transition and reward models. However, classical BRL methods assume known forms of transition and reward models. While recent deep BRL methods incorporate model learning to address this, applying neural networks directly to joint data and task parameters necessitates variational inference. This often yields indistinct task representations, compromising the resulting BRL policies. To overcome these limitations, we introduce **Generalised Linear Models in Deep Bayesian RL with Learnable Basis Functions (GLiBRL)**. Our approach features fully tractable Bayesian inference over task parameters and model noise, alongside exact marginal likelihood evaluation for learning transition and reward models. The permutation-invariant nature of exact Bayesian inference in GLiBRL enables seamless integration with both on-policy and off-policy RL algorithms. We further show that GLiBRL admits a closed-form relationship between the $\mathcal{L}_2$ distance of its task representations and empirical kernel-based correspondence between task samples, which is to our knowledge the first such structural result for online deep BRL. GLiBRL is compared against representative and recent Meta-RL methods, and improves state-of-the-art performance on both MuJoCo and MetaWorld benchmarks by up to $1.8\times$.

1 Introduction

Reinforcement Learning (RL) algorithms possess significant potential to enable autonomous, intelligent robotic control. However, standard RL algorithms typically do not account for variations in transition and reward dynamics, causing them to generalise poorly to unseen tasks where the underlying environmental models deviate from those encountered during training. Bayesian Reinforcement Learning (BRL), a special case of Meta-Reinforcement Learning (Meta-RL), offers an effective framework to overcome this limitation.

Instead of ignoring environmental variations, BRL explicitly accounts for them by assuming distributions over the transition and reward models, continually performing Bayesian inference on these parameters [11]. The resulting parameter distributions implicitly encode varying task dynamics. To solve these BRL problems, many classical methods employ planners [15, 27] to search for Bayes-optimal policies. However, these methods scale poorly and require explicit prior knowledge regarding the structural forms of the transition and reward models, severely restricting their generalisation across diverse tasks.

To address such rigidity, recent deep BRL methods [28, 42] enable model learning by optimising the marginal likelihood of the observed data. Unfortunately, the direct application of neural networks to the joint space of data and task parameters renders exact Bayesian inference intractable, preventing direct optimisation of the exact marginal likelihood. Consequently, these methods rely on variational

inference to optimise the Evidence Lower Bound (ELBO). This introduces challenges such as high-variance Monte Carlo estimates, amortisation gaps [4] and posterior collapse [3, 5]. In the context of BRL, these issues often prevent the extraction of meaningful and distinctive task parameters which are crucial for effective BRL.

To alleviate these issues, we propose **Generalised Linear Models in Deep Bayesian RL with Learnable Basis Functions (GLiBRL)**. GLiBRL enforces a generalised linear relation between task parameters and data features computed via learnable basis functions. This architecture circumvents variational inference entirely, permitting exact posterior inference and the computation of a closed-form marginal likelihood. Furthermore, by performing exact inference over the model noise, GLiBRL naturally generalises previous works [18] while reducing the error of predictions in unseen tasks. The permutation-invariance of GLiBRL induced by the exact Bayesian update also allows seamless integration with both on- and off-policy RL algorithms. Moreover, GLiBRL admits a closed-form identity relating $\mathcal{L}_2$ distances of task representations to kernel-based correspondence between task samples, with the kernels induced by the learnt basis functions. To our knowledge, this is the first such structural result for online deep BRL/Meta-RL: representations are not arbitrary embeddings but are tied directly to sample-level differences in the feature space the model itself learns.

We evaluate GLiBRL on the MuJoCo [34] locomotion benchmark and the challenging MetaWorld [41, 23] ML10/45 benchmarks. Compared against a diverse suite of Meta-RL baselines, including PEARL [28], VariBAD [42], SDVT [22], RL² [37, 7], MAML [9], AMAGO-v2 [13], TrMRL [24] and ECET [30], GLiBRL has demonstrated the state-of-the-art performance across both locomotion and manipulation domains. Specifically, GLiBRL achieves improvement by up to $1.8\times$ in return in MuJoCo, and by up to $1.1\times$ in success rate in MetaWorld.

2 Related Work

Bayesian Last Layers. GLiBRL is most relevant to ALPaCA [18], which introduces the concept of performing Bayesian inference only on the output layer of neural networks. ALPaCA provides an efficient, online Bayesian linear regression framework that also employs learnable basis functions. However, ALPaCA, alongside its subsequent extensions [19], was not formulated as a BRL method. While related work such as CAMELiD [17] employs similar controllers for policy computation, they rely on the restrictive assumption of fully known reward functions. GLiBRL generalises ALPaCA and CAMELiD in two ways: (1) whereas ALPaCA and CAMELiD assume a *known* noise in the likelihood function, GLiBRL performs exact Bayesian inference over the model noise, (2) GLiBRL operates seamlessly in online BRL settings with unknown rewards, whereas ALPaCA and CAMELiD focus on offline problems with simple, known rewards. As we will show in Section 5, the assumption of known noise induces predictive errors in both transition and reward models.

Reinforcement Learning. Standard RL methods are broadly categorised into model-free and model-based approaches. In this work, we use model-free algorithms, specifically Proximal Policy Optimization (PPO) [29] and Soft Actor-Critic (SAC) [16], to learn the BRL policies, aligning with a substantial portion of the Meta-RL literature. Furthermore, as GLiBRL learns transition and reward models, it is naturally compatible with model-based methods.

Hidden-Parameter MDPs. Hidden-Parameter MDPs (HiP-MDPs) [6] provide a framework for parametric Bayesian Reinforcement Learning. Initially modelled using Gaussian Processes (GPs) [6], HiP-MDPs were subsequently scaled by replacing GPs with Bayesian Neural Networks (BNNs) [21]. However, updating BNN weights at test-time has proved inefficient [38]. While follow-up work mitigated such inefficiency by fixing the test-time BNN weights and exclusively optimising task parameters [39], doing so strips away the true Bayesian features of the framework. Optimising task parameters while fixing BNN weights is mathematically equivalent to performing Maximum Likelihood Estimation (MLE), diverting the agent from Bayes-optimal exploration strategies (a limitation similarly noted by [42]). In contrast, recent parametric deep BRL methods [28, 26, 42, 22], including GLiBRL, are considered orthogonal to this line of work, as they do not assume known forms of reward functions and perform (approximate) Bayesian inference directly on task parameters rather than the neural network weights.

Classical Bayesian Reinforcement Learning. Classical BRL methods assume known forms of transitions and rewards. Early approaches formulated BRL as a Partially Observable MDP (POMDP) [20] utilising sampling-based offline solvers [27], while later methods introduced online, tree-

based solvers leveraging posterior sampling [33, 25] for efficiency [15]. These methods plan to find approximately optimal policies, which is orthogonal to the direct policy learning in GLiBRL. GLiBRL shares the idea of using generalised linear models with [35], while it differs fundamentally in that [35] *specifies* the basis functions, instead of learning them as in GLiBRL. Hand-crafting even a simple non-linear basis function, e.g., quadratic function, yields an $O(d^2)$ dimensional feature space, where d is the dimension of the raw input. As detailed in Section 4, performing online Bayesian inference scales at least quadratically with respect to the feature space dimension, meaning a prohibitive $O(d^4)$ complexity is required for a single inference step. In contrast, by *learning* basis functions, GLiBRL enables the use of compact low-dimensional feature spaces that capture non-linearities while preserving scalability.

Meta-Reinforcement Learning. Meta-Reinforcement Learning (Meta-RL) aims to learn policies from *seen* tasks that are capable of adapting to *unseen* tasks following similar task distributions [2]. According to [2], Meta-RL methods can be categorised as **(1) parameterised policy gradient (PPG)** methods (e.g., [9, 40, 10]) that learn by performing meta-policy gradients on the meta-parameterised policy; **(2) black-box methods** (e.g., [7, 37, 24, 13, 30]) that learn from summaries of histories and **(3) task-inference methods** (e.g., [28, 42, 22]) that learn from the belief of the task parameters (i.e., θ_T, θ_R in GLiBRL). Most of the deep BRL methods can be viewed as task inference methods, hence a subset of Meta-RL. BRL differs from other Meta-RL methods in that it performs Bayesian inference on task parameters, affording advantages such as uncertainty quantification. As a result, BRL methods have natural integrations with model-based algorithms, holding significance to research fields such as risk-aware control and planning under uncertainty.

3 Background

3.1 Markov Decision Processes and Reinforcement Learning

Markov Decision Processes (MDPs) are 5-tuples defined as $\mathcal{M} = (\mathcal{S}, \mathcal{A}, R, T, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $R(s_t, a_t, s_{t+1}, r_{t+1}) = p(r_{t+1}|s_t, a_t, s_{t+1})$ is the *reward* function, $T(s_t, a_t, s_{t+1}) = p(s_{t+1}|s_t, a_t)$ is the *transition* function, and $\gamma \in [0, 1]$ a discount factor. In the above definition, $s_t \in \mathcal{S}, s_{t+1} \in \mathcal{S}, a_t \in \mathcal{A}$ and $r_{t+1} \in \mathbb{R}$. The goal of solving an MDP is to find a policy $\pi(a_t|s_t) : \mathcal{S} \rightarrow \mathcal{A}$ that maximises the expected return over a finite horizon $H > 0$, given by $\mathcal{J}(\pi) = \mathbb{E}_\pi[\sum_{t=0}^H \gamma^t r_{t+1}]$. Standard Reinforcement Learning (RL) problems optimise this exact objective; however, they assume the transition dynamics T , the reward function R , or both are unknown and must be learnt from data.

3.2 Bayes-Adaptive MDPs and Bayesian Reinforcement Learning

Bayes-Adaptive MDP (BAMDP) [8, 11] is a Bayesian framework for RL. Unlike standard MDPs, BAMDPs assume transition and/or reward functions are parameterised by independent *unknown* parameters $\theta_T \in \Theta_T$ and/or $\theta_R \in \Theta_R$, respectively¹. Assume $p(\theta_T) \in \mathcal{B}_T$ and $p(\theta_R) \in \mathcal{B}_R$, a BAMDP maintains a joint task distribution (or *belief*²) $b_t = p_t(\theta_T, \theta_R) \in \mathcal{B}_T \times \mathcal{B}_R$ at timestep t that is updated to posteriors $b_{t+1} = p_{t+1}(\theta_T, \theta_R)$ via Bayesian inference when a new sample $(s_t, a_t, s_{t+1}, r_{t+1})$ is observed, i.e., $b_{t+1} = \tau(b_t, s_t, a_t, s_{t+1}, r_{t+1})$ where τ updates the belief.

To efficiently use existing MDP frameworks, beliefs can be absorbed into the original state space to form hyper-states $\mathcal{S}^+ = \mathcal{S} \times \mathcal{B}_T \times \mathcal{B}_R$. Hence, BAMDPs can be defined as 5-tuple $(\mathcal{S}^+, \mathcal{A}, R^+, T^+, \gamma)$ MDPs, where

$$\begin{aligned} T^+(s_t^+, a_t, s_{t+1}^+, r_{t+1}) &= p(s_{t+1}, b_{t+1}|s_t, b_t, a_t, r_{t+1}) \\ &= \mathbb{E}_{\theta_T \sim b_t} [p(s_{t+1}|s_t, a_t, \theta_T)] \cdot \delta(b_{t+1} = \tau(b_t, s_t, a_t, s_{t+1}, r_{t+1})) \\ &= \mathbb{E}_{\theta_T \sim b_t} [p(s_{t+1}|s_t, a_t, \theta_T)] \cdot \delta(b_{t+1} = p_{t+1}(\theta_T, \theta_R)) \end{aligned} \quad (1)$$

$$R^+(s_t^+, a_t, s_{t+1}^+, r_{t+1}) = p(r_{t+1}|s_t, b_t, a_t, s_{t+1}, b_{t+1}) = \mathbb{E}_{\theta_R \sim b_{t+1}} [p(r_{t+1}|s_t, a_t, s_{t+1}, \theta_R)] \quad (2)$$

The hyper-transition function (Equation 1) consists of the θ_T -parameterised expected regular MDP transition and a deterministic posterior update specified by the Dirac delta function $\delta(b_{t+1} =$

¹Throughout, we abuse the notation θ_T (resp. θ_R) for both the random variable and its realisation; context disambiguates.

²The term *belief* is taken from POMDPs [20], defined as distribution over unknown states. BAMDPs can be viewed as special cases of POMDPs where the unknown states are task parameters θ_T and/or θ_R .

$p_{t+1}(\theta_T, \theta_R)$). The hyper-reward function consists of the θ_R -parameterised expected regular MDP reward function. Accordingly, the expected return to maximise becomes $\mathcal{J}^+(\pi^+) = \mathbb{E}_{T^+, \pi^+, R^+}[\sum_{t=0}^{H^+} \gamma^t r_{t+1}]$, where $H^+ > 0$ is the BAMDP horizon, and $\pi^+ : \mathcal{S}^+ \rightarrow \mathcal{A}$ is the policy of BAMDPs³. Problems that require solving BAMDPs are named Bayesian Reinforcement Learning (BRL). Beyond improving generalisation, BRL offers a principled mechanism for addressing the exploration–exploitation trade-off [11].

Classical BRL methods [27, 15, 35] often assume the exact functional forms of T^+ and R^+ models are known a priori, despite being parameterised by unknown parameters. These methods have limited flexibility, as an incorrect assumption of the forms (e.g., assuming linear dynamics while the ground truth is quadratic) may lead to significant underfit. To relax these assumptions, Hidden-Parameter MDPs (HiP-MDPs) [6, 21, 39] learn the forms of models by performing Bayesian inference on the weights of neural networks. However, these methods scale poorly [38] and assume known forms of reward functions. Conversely, recent deep BRL methods [26, 17, 28, 42] achieve scalability by applying standard neural networks while retaining Bayesian properties through (approximate) inference directly on the task parameters θ_T, θ_R . GLiBRL aligns with this paradigm, accommodating unknown form of reward functions.

To contextualise our method, we formalise model learning in the recent deep BRL setting. The agent interacts with a set of MDPs with unknown transition dynamics and/or reward functions. A single sample at timestep t is denoted as $c_t := (s_t, a_t, s_{t+1}, r_{t+1})$. Within a given MDP $\mathcal{M}$, we define the collection of N samples as $\mathcal{C}^{\mathcal{M}} := \{c_t\}_{t=1}^N$. During training, assuming access to a batch of M MDPs $\{\mathcal{M}_i\}_{i=1}^M$, the resulting joint dataset is given by $\mathcal{C} = \bigcup_{i=1}^M \mathcal{C}^{\mathcal{M}_i}$. The objective in most recent deep BRL methods [28, 26, 42] is to maximise the marginal log-likelihood of this joint data

$$\log p_{\zeta, \phi_T, \phi_R}(\mathcal{C}) = \sum_{i=1}^M \log p_{\zeta, \phi_T, \phi_R}(\mathcal{C}^{\mathcal{M}_i}) = \sum_{i=1}^M \log \int p_{\zeta}(\theta_T, \theta_R) p_{\phi_T, \phi_R}(\mathcal{C}^{\mathcal{M}_i} | \theta_T, \theta_R) d\theta_T d\theta_R \quad (3)$$

where ζ is an optional neural network parameterising the prior $p(\theta_T, \theta_R)$. The likelihood of an individual task factors as $p_{\phi_T, \phi_R}(\mathcal{C}^{\mathcal{M}_i} | \theta_T, \theta_R) \propto \prod_{t=1}^N p_{\phi_T}(s_{t+1} | s_t, a_t, \theta_T) p_{\phi_R}(r_{t+1} | s_t, a_t, s_{t+1}, \theta_R)$, where ϕ_T and ϕ_R denote neural networks to learn the forms of transition and reward functions, respectively.

To facilitate optimisation, p_{ϕ_T} and p_{ϕ_R} are typically modelled as Gaussians with means and diagonal covariances determined by the output of neural networks ϕ_T, ϕ_R . The prior $p_{\zeta}(\theta_T, \theta_R)$ is also assumed to be a Gaussian. However, note that even with these distributional simplifications, the exact marginal likelihood in Equation 3 remains intractable. Because the neural network introduces high non-linearity between task parameters θ_T, θ_R and the data, the analytical integration is impossible. Consequently, these methods are forced to resort to variational inference, optimising the Evidence Lower Bound (ELBO) as a tractable surrogate objective (proof see Appendix A.5):

$$\log p_{\zeta, \phi_T, \phi_R}(\mathcal{C}) \geq \sum_{i=1}^M \mathbb{E}_q [\log p_{\phi_T, \phi_R}(\mathcal{C}^{\mathcal{M}_i} | \theta_T, \theta_R)] - D_{KL}(q(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i}) || p_{\zeta}(\theta_T, \theta_R)) \quad (4)$$

where $D_{KL}(\cdot || \cdot)$ is the KL-divergence, and $q(\cdot)$ is an approximate Gaussian posterior of θ_T, θ_R .

Variational inference enables model learning, but it introduces optimisation challenges such as high-variance Monte Carlo estimates, amortisation gaps [4] and posterior collapse [3, 5]. These issues frequently prevent learning meaningful and distinctive task representations. Different from other tasks where the quality of learnt representations might be less important, BRL policies rely heavily on continually updated task representations to compute actions. Indistinctive representations will substantially harm the performance of BRL policies. As demonstrated in Appendix A.14, existing methods like VariBAD [42] suffer from posterior collapse, whereas our method successfully learns meaningful task representations.

The other challenge is to compute the approximate posterior $q(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i})$ directly on variable-length $\mathcal{C}^{\mathcal{M}_i}$. The most common approach is to use sequential models like RNNs or Transformers [36, 13, 24, 30] to summarise $\mathcal{C}^{\mathcal{M}_i}$. However, because these models are permutation-variant, directly combining them with sample-efficient off-policy algorithms like Soft Actor-Critic (SAC) [16] causes

³We will use BAMDP policy and BRL policy interchangeably when referring to π^+ .

a severe performance drop, as demonstrated by an ablation study in PEARL [28]. The other approach, used by PEARL, is a factored approximation $q(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i}) \approx \prod_{t=1}^N \mathcal{N}(\theta_T, \theta_R | g(\mathcal{C}^{\mathcal{M}_i}[t]))$, where $g(\cdot)$ is a neural network that takes the t -th sample in $\mathcal{C}^{\mathcal{M}_i}$ as the input and returns the mean and covariance of a Gaussian as the output. However, Bayes' rule reveals that $g(\cdot)$ must essentially predict $p(\theta_T, \theta_R)^{(1/N)-1} q(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i}[t])$. Because this target relies heavily on the length N , it creates shifting optimisation targets as the dataset grows, resulting in inaccurate posterior approximations. In contrast, GLiBRL completely circumvents the need to compute an approximate posterior in the first place; its exact Bayesian update is naturally permutation-invariant, enabling seamless integration with SAC, as we will demonstrate in Section 5.

4 Generalised Linear Models in Deep BRL with Learnable Basis Functions

4.1 Overview

Previous deep BRL methods rely on challenging variational inference and heavily approximated posterior updates, which often yield inaccurate task distributions and degrade policy performance. In contrast, GLiBRL employs generalised linear models with learnable basis functions to enable fully tractable, permutation-invariant posterior updates. Crucially, this inherent permutation invariance allows GLiBRL to seamlessly integrate with both on- and off-policy algorithms. Moreover, GLiBRL allows for the exact computation of the marginal log-likelihood, entirely bypassing the need to perform variational inference and evaluate an Evidence Lower Bound (ELBO). While the assumption of linearity in the parameter space might initially appear restrictive, the use of non-linear basis functions (parameterised by deep neural networks) ensures the transition and reward models retain their full, non-linear representational capacity. Furthermore, GLiBRL is the first online deep BRL/Meta-RL method with a closed-form identity relating $\mathcal{L}_2$ distances of task representations to kernel-based correspondence between task samples.

GLiBRL is an online algorithm whose training alternates between data collection and learning. During data collection, a batch of MDPs $\{\mathcal{M}_i\}_{i=1}^K$ is sampled from the task distribution. Samples $\{\mathcal{C}^{\mathcal{M}_i}\}_{i=1}^K$ are maintained for each MDP separately. Starting from state s , to collect samples (s, a, s', r) to be stored in $\mathcal{C}^{\mathcal{M}_i}$, GLiBRL sequentially performs the following at each timestep: (1) updates the current belief b to the exact posterior distribution $p_{\phi_T, \phi_R}(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i})$ (2) samples an action a from the BAMDP policy $\pi_{\psi}^+(a|s, b)$ where ψ is a neural network, and (3) executes a to observe the next state s' and reward r . Once sufficient samples are collected across tasks, GLiBRL samples a batch of samples $\mathcal{D}$ from the joint buffer $\mathcal{C} = \bigcup_i \mathcal{C}^{\mathcal{M}_i}$ and updates the BAMDP policy network ψ and transition/reward models ϕ_T, ϕ_R via gradient descent on $\mathcal{L}_{\text{policy}}$ and $\mathcal{L}_{\text{model}}$, respectively. The complete procedure is detailed in Algorithm 1.

The success of GLiBRL relies on solving three primary challenges: (1) formulating a tractable, sequentially updated posterior $p_{\phi_T, \phi_R}(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i})$, (2) deriving an ELBO-free objective $\mathcal{L}_{\text{model}}$ for updating ϕ_T, ϕ_R , and (3) constructing theoretically grounded task representations for learning the policy π_{ψ}^+ . We will detail these solutions in Section 4.2, Section 4.3, and Section 4.4, respectively.

4.2 Tractable and Sequentially Updated Posterior Distributions

To derive the exact posterior $p_{\phi_T, \phi_R}(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i})$, we must explicitly define the prior $p(\theta_T, \theta_R)$ and the likelihood $p_{\phi_T, \phi_R}(\mathcal{C}^{\mathcal{M}_i} | \theta_T, \theta_R)$. Let the task data be denoted compactly as $\mathcal{C}^{\mathcal{M}_i} = (\mathbf{S}_i \in \mathbb{R}^{N \times D_S}, \mathbf{A}_i \in \mathbb{R}^{N \times D_A}, \mathbf{S}'_i \in \mathbb{R}^{N \times D_S}, \mathbf{r}_i \in \mathbb{R}^{N \times 1})$, where N is the number of samples and D_S, D_A are the dimensions of the state and action space. Let the task variables be defined as $\theta_T = (T_{\mu} \in \mathbb{R}^{D_T \times D_S}, T_{\sigma} \in \mathbb{R}^{D_S \times D_S})$ and $\theta_R = (R_{\mu} \in \mathbb{R}^{D_R \times 1}, R_{\sigma} \in \mathbb{R}^{1 \times 1})$, where D_T, D_R are task dimensions. We formulate the prior as Normal-Wishart distributions:

$$p(\theta_T, \theta_R) = p(\theta_T) \cdot p(\theta_R) \quad (\text{Assumed independent } \theta_T, \theta_R) \quad (5)$$

$$p(\theta_T) = \mathcal{MN}(T_{\mu} | \mathbf{M}_T \in \mathbb{R}^{D_T \times D_S}, \mathbf{\Xi}_T^{-1} \in \mathbb{R}^{D_T \times D_T}, T_{\sigma}) \cdot \mathcal{W}(T_{\sigma}^{-1} | \mathbf{\Omega}_T^{-1} \in \mathbb{R}^{D_S \times D_S}, \nu_T > D_S - 1) \quad (6)$$

$$p(\theta_R) = \mathcal{MN}(R_{\mu} | \mathbf{M}_R \in \mathbb{R}^{D_R \times 1}, \mathbf{\Xi}_R^{-1} \in \mathbb{R}^{D_R \times D_R}, R_{\sigma}) \cdot \mathcal{W}(R_{\sigma}^{-1} | \mathbf{\Omega}_R^{-1} \in \mathbb{R}^{1 \times 1}, \nu_R > 0) \quad (7)$$

Here, $\mathcal{W}(\mathbf{W} | \mathbf{X}, \nu)$ defines a Wishart distribution on positive definite random matrix $\mathbf{W}$ with scale $\mathbf{X}$ and degrees of freedom ν , and $\mathcal{MN}(\mathbf{W} | \mathbf{X}, \mathbf{Y}, \mathbf{Z})$ defines a matrix normal distribution with random

matrix $\mathbf{W}$, mean $\mathbf{X}$, row covariance $\mathbf{Y}$ and column covariance $\mathbf{Z}$. Normal-Wishart priors enable fully tractable Bayesian inference over the model noises T_σ, R_σ , generalising previous work [17, 18].

We then define the Matrix-Normal likelihood function:

$$p_{\phi_T, \phi_R}(\mathcal{C}^{\mathcal{M}_i} | \theta_T, \theta_R) = p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i, \theta_T) \cdot p_{\phi_R}(\mathbf{r}_i | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i, \theta_R) \cdot p(\mathbf{S}_i, \mathbf{A}_i) \quad (8)$$

$$p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i, \theta_T) = \mathcal{MN}(\mathbf{S}'_i | \mathbf{C}_T T_\mu, \mathbf{I}_N, T_\sigma) \quad (9)$$

$$p_{\phi_R}(\mathbf{r}_i | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i, \theta_R) = \mathcal{MN}(\mathbf{r}_i | \mathbf{C}_R R_\mu, \mathbf{I}_N, R_\sigma) \quad (10)$$

$$\mathbf{C}_T \in \mathbb{R}^{N \times D_T} = \phi_T(\mathbf{S}_i, \mathbf{A}_i) \quad \mathbf{C}_R \in \mathbb{R}^{N \times D_R} = \phi_R(\mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i) \quad (11)$$

The term $p(\mathbf{S}_i, \mathbf{A}_i)$ acts as a constant irrelevant to the optimisation, while $\mathbf{C}_T$ and $\mathbf{C}_R$ ⁴ represent data features generated by neural networks ϕ_T, ϕ_R acting as expressive and learnable basis functions. Unlike other deep BRL methods [28, 42] that apply neural networks directly to the joint space of data and task parameters, GLiBRL strictly separates them. By restricting the linearisation exclusively to the mapping between the feature space and the Bayesian parameters, we preserve exact mathematical tractability.

Because the Normal-Wishart prior is conjugate to the Matrix-Normal likelihood, the resulting posteriors also follow Normal-Wishart distributions (proof in Appendix A.6)

$$p_{\phi_T, \phi_R}(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i}) = p_{\phi_T}(\theta_T | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i) \cdot p_{\phi_R}(\theta_R | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i, \mathbf{r}_i) \quad (12)$$

$$p_{\phi_T}(\theta_T | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i) = \mathcal{MN}(T_\mu | \mathbf{M}'_T, \mathbf{\Xi}'_T^{-1}, T_\sigma) \cdot \mathcal{W}(T_\sigma^{-1} | \mathbf{\Omega}'_T^{-1}, \nu'_T) \quad (13)$$

$$p_{\phi_R}(\theta_R | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i, \mathbf{r}_i) = \mathcal{MN}(R_\mu | \mathbf{M}'_R, \mathbf{\Xi}'_R^{-1}, R_\sigma) \cdot \mathcal{W}(R_\sigma^{-1} | \mathbf{\Omega}'_R^{-1}, \nu'_R) \quad (14)$$

where

$$\begin{aligned} \mathbf{M}'_T &= \mathbf{\Xi}'_T^{-1} [\mathbf{C}_T^T \mathbf{S}' + \mathbf{\Xi}_T \mathbf{M}_T] & \mathbf{M}'_R &= \mathbf{\Xi}'_R^{-1} [\mathbf{C}_R^T \mathbf{r} + \mathbf{\Xi}_R \mathbf{M}_R] \\ \mathbf{\Xi}'_T &= \mathbf{C}_T^T \mathbf{C}_T + \mathbf{\Xi}_T & \mathbf{\Xi}'_R &= \mathbf{C}_R^T \mathbf{C}_R + \mathbf{\Xi}_R \\ \mathbf{\Omega}'_T &= \mathbf{\Omega}_T + \mathbf{S}'^T \mathbf{S}' & \mathbf{\Omega}'_R &= \mathbf{\Omega}_R + \mathbf{r}^T \mathbf{r} \\ &+ \mathbf{M}_T^T \mathbf{\Xi}_T \mathbf{M}_T - \mathbf{M}_T'^T \mathbf{\Xi}'_T \mathbf{M}'_T & &+ \mathbf{M}_R^T \mathbf{\Xi}_R \mathbf{M}_R - \mathbf{M}_R'^T \mathbf{\Xi}'_R \mathbf{M}'_R \\ \nu'_T &= \nu_T + N & \nu'_R &= \nu_R + N \end{aligned} \quad (15)$$

During online execution, the agent must sequentially update priors to posteriors with each newly observed sample $c_t = \{s_t, a_t, s'_t, r_{t+1}\}$ for arbitrary timestep t . Naively computing Equation 15 requires inversion of $\mathbf{\Xi}'_T$ and $\mathbf{\Xi}'_R$, which scale at $\mathcal{O}(D_T^3)$ and $\mathcal{O}(D_R^3)$, respectively. Applying the matrix inversion lemma yields

$$\mathbf{\Xi}'_T^{-1} = (\mathbf{C}_T^T \mathbf{C}_T + \mathbf{\Xi}_T)^{-1} = \mathbf{\Xi}_T^{-1} - \mathbf{\Xi}_T^{-1} \mathbf{C}_T^T (\mathbf{I}_N + \mathbf{C}_T \mathbf{\Xi}_T^{-1} \mathbf{C}_T^T)^{-1} \mathbf{C}_T \mathbf{\Xi}_T^{-1} \quad (16)$$

$$\mathbf{\Xi}'_R^{-1} = (\mathbf{C}_R^T \mathbf{C}_R + \mathbf{\Xi}_R)^{-1} = \mathbf{\Xi}_R^{-1} - \mathbf{\Xi}_R^{-1} \mathbf{C}_R^T (\mathbf{I}_N + \mathbf{C}_R \mathbf{\Xi}_R^{-1} \mathbf{C}_R^T)^{-1} \mathbf{C}_R \mathbf{\Xi}_R^{-1} \quad (17)$$

With a single sample, $\mathbf{C}_T \in \mathbb{R}^{1 \times D_T}$ and $\mathbf{C}_R \in \mathbb{R}^{1 \times D_R}$. Hence, $(\mathbf{I}_N + \mathbf{C}_T \mathbf{\Xi}_T^{-1} \mathbf{C}_T^T)^{-1}$ and $(\mathbf{I}_N + \mathbf{C}_R \mathbf{\Xi}_R^{-1} \mathbf{C}_R^T)^{-1}$ are reduced to reciprocals of scalars. By caching the inverted matrices, this reduces the entire online sequential update complexity to $\mathcal{O}(\max(D_S^2, D_T^2))$ for transitions and $\mathcal{O}(D_R^2)$ for rewards, enabling efficient data collection.

4.3 ELBO-free Model Updates

The fully tractable posterior update yields a closed-form marginal likelihood, which contributes to the model loss $\mathcal{L}_{\text{model}}$ for optimising model parameters ϕ_T, ϕ_R . Dropping the optional neural network ζ , Equation 3 can be written analytically as

$$\begin{aligned} \log p_{\phi_T, \phi_R}(\mathcal{C}) &= \sum_{i=1}^M \log \int p(\theta_T) p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i, \theta_T) d\theta_T \\ &+ \sum_{i=1}^M \log \int p(\theta_R) p_{\phi_R}(\mathbf{r}_i | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i, \theta_R) d\theta_R + \text{const.} \end{aligned} \quad (18)$$

⁴Dependence on i of $\mathbf{C}_T, \mathbf{C}_R$ is omitted for conciseness. This also applies to $\mathbf{M}'_T, \mathbf{\Xi}'_T, \mathbf{\Omega}'_T, \mathbf{M}'_R, \mathbf{\Xi}'_R, \mathbf{\Omega}'_R$.

Substituting accordingly with equations in Section 4.2, we reduce the integral to a closed-form marginal log-likelihood (proof in Appendix A.7):

$$\log p_{\phi_T, \phi_R}(\mathcal{C}) = -\frac{1}{2} \sum_{i=1}^M \left(D_S \log |\Xi'_T| + \nu'_T \log \left| \frac{1}{2} \Omega'_T \right| + \log |\Xi'_R| + \nu'_R \log \left| \frac{1}{2} \Omega'_R \right| \right) + \text{const.} \quad (19)$$

To optimise ϕ_T and ϕ_R , Equation 19 is maximised with respect to features $\mathbf{C}_T$ and $\mathbf{C}_R$. To prevent overfitting, we apply squared Frobenius norms $\|\mathbf{C}_T\|_F^2$ and $\|\mathbf{C}_R\|_F^2$ to Equation 19 as regularisations, the effect of which is discussed in Appendix A.16. The regularised model loss is defined as

$$\mathcal{L}_{\text{model}} := -\log p_{\phi_T, \phi_R}(\mathcal{C}) + \lambda_T \|\mathbf{C}_T\|_F^2 + \lambda_R \|\mathbf{C}_R\|_F^2 \quad (20)$$

where $\lambda_T > 0$ and $\lambda_R > 0$ are hyperparameters. Note that $\mathcal{L}_{\text{model}}$ can be directly minimised with gradient descent, without any variational inference and the evaluation of the ELBO term.

4.4 Task Representation in Policy Networks

The final component in GLiBRL involves integrating beliefs, which define tasks, into the policy network $\pi_{\psi}^+(a|s, b)$ to drive decision-making and sample collection. We concisely represent this belief using the deterministic means of the task posteriors:

$$\pi_{\psi}^+(a|s, b) \approx \pi_{\psi}^+(a|s, f(\mathbf{M}'_T, \mathbf{M}'_R)) \quad (21)$$

where $f: \mathbb{R}^{D_T \times D_S} \times \mathbb{R}^{D_R \times 1} \rightarrow \mathbb{R}^{D_P \times 1}$ and D_P is the dimension of the belief representation. Specifically, assuming non-zero posterior means, we utilise the normalised and concatenated transition and reward parameter means

$$f_{T,R}(\mathbf{M}'_T, \mathbf{M}'_R) = \left[\frac{\text{tril}(\mathbf{M}'_T \mathbf{M}_T'^T)^T}{\|\text{tril}(\mathbf{M}'_T \mathbf{M}_T'^T)\|_2} \frac{\mathbf{M}'_R}{\|\mathbf{M}'_R\|_2} \right]^T \quad (22)$$

where $\text{tril}(\cdot)$ retrieves the lower triangle of a matrix and flattens it into a vector. Let samples from two MDPs $\mathcal{M}_1, \mathcal{M}_2$ be $\mathcal{C}^{\mathcal{M}_1}$ and $\mathcal{C}^{\mathcal{M}_2}$, where

$$\mathcal{C}^{\mathcal{M}_1} = (\mathbf{S}_1, \mathbf{A}_1, \mathbf{S}'_1, \mathbf{r}_1) \quad \mathcal{C}^{\mathcal{M}_2} = (\mathbf{S}_2, \mathbf{A}_2, \mathbf{S}'_2, \mathbf{r}_2)$$

Given GLiBRL task posterior means $\mathbf{M}_T^{1'}, \mathbf{M}_R^{1'}, \mathbf{M}_T^{2'}, \mathbf{M}_R^{2'}$, assuming shared prior means $\mathbf{M}_T = \mathbf{0}$, $\mathbf{M}_R = \mathbf{0}^5$ and covariances Ξ_T, Ξ_R , where the posterior update follows Equation 15. The following equivalence strictly holds:

$$\begin{aligned} & \left\| f_{T,R}(\mathbf{M}_T^{1'}, \mathbf{M}_R^{1'}) - f_{T,R}(\mathbf{M}_T^{2'}, \mathbf{M}_R^{2'}) \right\|_2^2 \\ &= 4 - 2 \cdot (\text{sCMS}_{k_{TT}}^{w_{TT}}(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) + \text{sCMS}_{k_R}^{w_R}(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2})) \end{aligned} \quad (23)$$

where sCMS defines the empirical signed-measure Cosine Mean Similarity between $\mathcal{C}^{\mathcal{M}_1}$ and $\mathcal{C}^{\mathcal{M}_2}$ measured in the feature space. Note that alternative mapping functions exhibit similar theoretical properties (detailed in Appendix A.8). For instance, the unnormalised variant of $f_{T,R}$,

i.e., $\widetilde{f_{T,R}}(\mathbf{M}'_T, \mathbf{M}'_R) = [\text{tril}(\mathbf{M}'_T \mathbf{M}_T'^T)^T \mathbf{M}'_R]^T$ maps to the empirical signed-measure Maximum Mean Discrepancy (sMMD):

$$\left\| \widetilde{f_{T,R}}(\mathbf{M}_T^{1'}, \mathbf{M}_R^{1'}) - \widetilde{f_{T,R}}(\mathbf{M}_T^{2'}, \mathbf{M}_R^{2'}) \right\|_2^2 = \text{sMMD}_{k_{TTR}}^{w_{TTR}}(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) \quad (24)$$

Empirically, we find that $f_{T,R}$ leads to a more stable performance. The definitions of both sCMS and sMMD, all related proofs, the forms of kernel functions k_{TT}, k_R, k_{TTR} and weighting functions w_{TT}, w_R, w_{TTR} are in Appendix A.9.

Overall, Equation 23 and Equation 24 tie task representation distances to the similarity of task samples in the feature space induced by ϕ_T and ϕ_R learnt from maximising the marginal log-likelihood (Equation 20). Task samples dissimilar in the feature space must correspond to different task representations, while similar task samples yield similar task representations. The empirical results in Appendix A.13 visualise this clearly.

⁵Zero prior means are unnecessary and just for notational conciseness. For example, $\mathbf{C}_T^T \mathbf{S}' + \Xi_T \mathbf{M}_T = [\mathbf{C}_T^T \quad \mathbf{I}] [\mathbf{S}'^T \quad \mathbf{M}_T^T \Xi_T]^T = \widetilde{\mathbf{C}}_T^T \widetilde{\mathbf{S}}'$ and similar theoretical properties hold trivially. A non-zero $\mathbf{M}_T$ also guarantees non-zero denominator in Equation 22.

5 Experiments

We evaluate GLiBRL on locomotion tasks from the MuJoCo benchmark [34] and manipulation tasks from the MetaWorld [41, 23] benchmark. For MuJoCo, following standard Meta-RL evaluation protocols, we assess performance on `HalfCheetahDir`, `AntDir`, and `HalfCheetahVel`. In these environments, the agent must adapt its policy from the observed reward to run in the randomly determined forward/backward direction or at a targeted velocity. For MetaWorld, which is considered one of the most challenging meta-RL benchmarks, we focus on its most complex subsets: Meta-Learning 10 (ML10) and Meta-Learning 45 (ML45). ML10 and ML45 consist of 10 and 45 training tasks, respectively, alongside 5 held-out testing tasks where the agent must adapt to both unseen transition and reward functions. To ensure fair comparison, we use MetaWorld-V3 [23].

We benchmark GLiBRL against a diverse set of Meta-RL baselines spanning all three main categories: (1) PPG-based methods: **MAML** [9], (2) black-box methods: **RL²** [7], **AMAGO-v2** [13], **TrMRL** [24] and **ECET** [30], and (3) task-inference (deep BRL) methods: **PEARL** [28], **VariBAD** [42] and **SDVT** [22]. Specifically, we compare with RL², TrMRL, PEARL, VariBAD in the locomotion tasks in MuJoCo, and MAML, RL², VariBAD, AMAGO-v2, SDVT, TrMRL and ECET in the manipulation tasks in MetaWorld. ⁶ Full implementation details are provided in Appendix A.10.

Across all experiments, we focus on comparing the zero-shot test-time performance, which is critical to Meta-RL aiming to adapt with minimal data [2]. For MuJoCo experiments (Section 5.1), we follow the setting in PEARL and test GLiBRL with SAC [16] (Algorithm 2) over five random seeds. Both PEARL and GLiBRL are trained with $1e6$ total steps, while others are trained with $1e7$ total steps. For MetaWorld experiments (Section 5.2), we follow [23] and run GLiBRL with PPO [29] (Algorithm 3) over ten random seeds. ML10 and ML45 experiments are run for $2e7$ steps and $5e7$ steps, respectively. Beyond quantitative evaluations, we qualitatively validate our theoretical guarantees regarding structural correspondence by visualising the learnt task representations (Section 5.3). Finally, using ML10, we conduct ablation studies to empirically validate the necessity of placing a Wishart distribution on model noises (Section 5.4). Each experiment is run on a single A100 GPU⁷.

5.1 MuJoCo Results

As shown in the top row in Figure 1, GLiBRL combined with SAC demonstrates superior sample efficiency on MuJoCo locomotion tasks. Using only $1e6$ training steps, GLiBRL outperforms or matches all PPO-based baselines (RL², TrMRL, VariBAD) by up to $1.5\times$, achieving this with merely 10% of their step budget. Crucially, as discussed in Section 3, none of these PPO-based baselines can be trivially integrated with SAC, as they lack permutation invariance critical for off-policy methods [28]. Furthermore, GLiBRL outperforms SAC-based PEARL by up to $1.8\times$, which was previously considered a state-of-the-art method for MuJoCo locomotion tasks, highlighting the advantage of GLiBRL’s exact Bayesian inference over PEARL’s factored approximation.

5.2 MetaWorld Results

The bottom row of Figure 1 demonstrates GLiBRL’s state-of-the-art performance on MetaWorld manipulation tasks when integrated with PPO. In ML10, GLiBRL matches the performance of MAML and RL² while substantially outperforming all other methods⁸. In ML45 where the larger number of training tasks necessitates the stronger capability of identifying tasks, GLiBRL outperforms all evaluated baselines. Detailed per-task success rates for the top methods (GLiBRL and MAML) are in Appendix A.11 and Appendix A.12.

5.3 Qualitative Analysis of Task Identifiability

To empirically validate the structural correspondence guaranteed by GLiBRL (Section 4.4), we visualise the task representations for the `HalfCheetahVel` environment using 2D Principal Component

⁶Methods excluded from these comparisons either lack reported results or demonstrate uncompetitive performance. For example, MAML has return < 0 with $1e7$ steps in `HalfCheetahDir` [24]. PEARL has $< 3\%$ success rate in ML10 with $1e8$ steps [41]. We exclude ECET in MuJoCo for it not being open-source and reporting MuJoCo results in meta-steps.

⁷A100 is not mandatory. GLiBRL is runnable with ≤ 8 GB GPU memory, see Appendix A.18.

⁸In Appendix A.15, we show that GLiBRL reveals even **higher** success rates (**29%**) in ML10 when setting $D_T = 8$, at the cost of slightly higher predictive error.

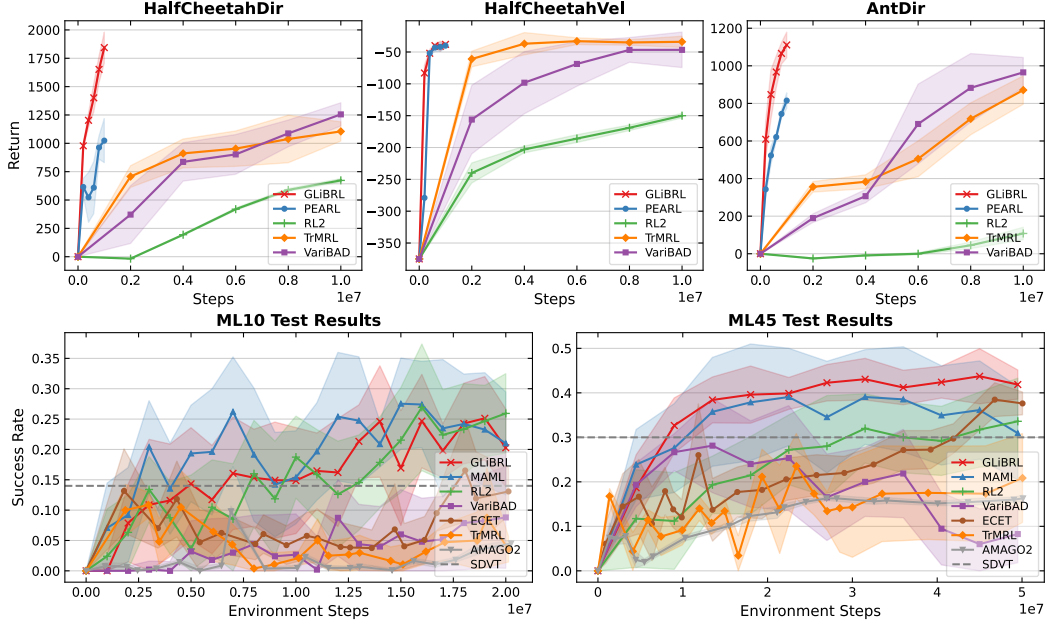


Figure 1: Test-time return and success rate in all benchmarks, where shaded regions denote 95% confidence intervals. In GLiBRL, $D_T = 16$ and $D_R = 256$ for MetaWorld tasks, and $D_R = 8$ for MuJoCo tasks which do not require inference in transition. In MuJoCo tasks, we run PEARL and GLiBRL for $1e6$ steps and other baselines for $1e7$ steps.

Analysis (PCA). As detailed in Appendix A.13, GLiBRL successfully disentangles the continuous target velocities into a highly structured, cleanly identifiable latent space after only $5e5$ training steps. In contrast, strong baselines such as TrMRL fail to achieve meaningful task separation even when allocated a $10\times$ larger training budget. This rapid and precise task identifiability visually confirms the advantage of our exact Bayesian update.

5.4 Ablation Studies

We conduct ablation studies demonstrating that performing explicit Bayesian inference over model noises mitigates the predictive errors in both transition and reward models. Full experimental results and mathematical derivations are deferred to Appendix A.16.

6 Conclusion

We introduced GLiBRL, a novel deep BRL framework that enables fully tractable inference over task parameters and efficient learning of basis functions via exact, ELBO-free marginal likelihood optimisation. By performing Bayesian inference over the model noise, GLiBRL reduces predictive errors. Furthermore, GLiBRL naturally guarantees permutation invariance, allowing seamless integration with both on- and off-policy RL algorithms. Crucially, we provided the first structural result in online deep BRL/Meta-RL establishing a closed-form identity between the $\mathcal{L}_2$ distance of task representations and kernel-based empirical correspondence of task samples in the learned feature space, under mild assumptions. Empirically, GLiBRL improves state-of-the-art performance across both the MuJoCo locomotion and the MetaWorld ML10/45 benchmarks by up to $1.8\times$.

Several promising directions for future work arise naturally. Because GLiBRL learns transition and reward models, it is well-suited for integration with model-based RL algorithms, which could further improve sample efficiency. However, deploying model-based planning requires frequent sampling from the learned models; currently, sampling from the high-dimensional Wishart distributions presents a computational bottleneck that warrants future optimisation. Alternatively, within the model-free paradigm, it is interesting to explore how to incorporate uncertainties within task representations. Directly feeding the covariances in the policy network will introduce high training instability.

References

- [1] Jacob Beck, Matthew Thomas Jackson, Risto Vuorio, and Shimon Whiteson. Hypernetworks in meta-reinforcement learning. In *Conference on Robot Learning*, pages 1478–1487. PMLR, 2023.
- [2] Jacob Beck, Risto Vuorio, Evan Zheran Liu, Zheng Xiong, Luisa Zintgraf, Chelsea Finn, and Shimon Whiteson. A survey of meta-reinforcement learning. *arXiv preprint arXiv:2301.08028*, 2023.
- [3] Samuel Bowman, Luke Vilnis, Oriol Vinyals, Andrew Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL conference on computational natural language learning*, pages 10–21, 2016.
- [4] Chris Cremer, Xuechen Li, and David Duvenaud. Inference suboptimality in variational autoencoders. In *International conference on machine learning*, pages 1078–1086. PMLR, 2018.
- [5] Bin Dai, Ziyu Wang, and David Wipf. The usual suspects? reassessing blame for vae posterior collapse. In *International conference on machine learning*, pages 2313–2322. PMLR, 2020.
- [6] Finale Doshi-Velez and George Konidaris. Hidden parameter markov decision processes: A semiparametric regression approach for discovering latent task parametrizations. In *IJCAI: proceedings of the conference*, volume 2016, page 1432, 2016.
- [7] Yan Duan, John Schulman, Xi Chen, Peter L Bartlett, Ilya Sutskever, and Pieter Abbeel. RI^2 : Fast reinforcement learning via slow reinforcement learning. *arXiv preprint arXiv:1611.02779*, 2016.
- [8] Michael O’Gordon Duff. *Optimal Learning: Computational procedures for Bayes-adaptive Markov decision processes*. University of Massachusetts Amherst, 2002.
- [9] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.
- [10] Chelsea Finn, Kelvin Xu, and Sergey Levine. Probabilistic model-agnostic meta-learning. *Advances in neural information processing systems*, 31, 2018.
- [11] Mohammad Ghavamzadeh, Shie Mannor, Joelle Pineau, Aviv Tamar, et al. Bayesian reinforcement learning: A survey. *Foundations and Trends® in Machine Learning*, 8(5-6):359–483, 2015.
- [12] Arthur Gretton, Karsten M Borgwardt, Malte J Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *The journal of machine learning research*, 13(1):723–773, 2012.
- [13] Jake Grigsby, Justin Sasek, Samyak Parajuli, Daniel Adebisi, Amy Zhang, and Yuke Zhu. Amago-2: Breaking the multi-task barrier in meta-reinforcement learning with transformers. *Advances in Neural Information Processing Systems*, 37:87473–87508, 2024.
- [14] Sebastian G Gruber, Pascal Tobias Ziegler, and Florian Buettner. Disentangling mean embeddings for better diagnostics of image generators. *arXiv preprint arXiv:2409.01314*, 2024.
- [15] Arthur Guez, David Silver, and Peter Dayan. Scalable and efficient bayes-adaptive reinforcement learning based on monte-carlo tree search. *Journal of Artificial Intelligence Research*, 48:841–883, 2013.
- [16] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.
- [17] James Harrison, Apoorva Sharma, Roberto Calandra, and Marco Pavone. Control adaptation via meta-learning dynamics. In *Workshop on Meta-Learning at NeurIPS*, volume 2018, 2018.

- [18] James Harrison, Apoorva Sharma, and Marco Pavone. Meta-learning priors for efficient online bayesian regression. In *International Workshop on the Algorithmic Foundations of Robotics*, pages 318–337. Springer, 2018.
- [19] James Harrison, John Willes, and Jasper Snoek. Variational bayesian last layers. *arXiv preprint arXiv:2404.11599*, 2024.
- [20] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101(1-2):99–134, 1998.
- [21] Taylor W Killian, Samuel Daulton, George Konidaris, and Finale Doshi-Velez. Robust and efficient transfer learning with hidden parameter markov decision processes. *Advances in neural information processing systems*, 30, 2017.
- [22] Suyoung Lee, Myungsik Cho, and Youngchul Sung. Parameterizing non-parametric meta-reinforcement learning tasks via subtask decomposition. *Advances in Neural Information Processing Systems*, 36:43356–43383, 2023.
- [23] Reginald McLean, Evangelos Chatzaroulas, Luc McCutcheon, Frank Röder, Tianhe Yu, Zhanpeng He, KR Zentner, Ryan Julian, JK Terry, Isaac Woungang, et al. Meta-world+: An improved, standardized, rl benchmark. *arXiv preprint arXiv:2505.11289*, 2025.
- [24] Luckeciano C Melo. Transformers are meta-reinforcement learners. In *international conference on machine learning*, pages 15340–15359. PMLR, 2022.
- [25] Ian Osband, Daniel Russo, and Benjamin Van Roy. (more) efficient reinforcement learning via posterior sampling. *Advances in Neural Information Processing Systems*, 26, 2013.
- [26] Christian Perez, Felipe Petroski Such, and Theofanis Karaletsos. Generalized hidden parameter mdps: Transferable model-based rl in a handful of trials. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 5403–5411, 2020.
- [27] Pascal Poupart, Nikos Vlassis, Jesse Hoey, and Kevin Regan. An analytic solution to discrete bayesian reinforcement learning. In *Proceedings of the 23rd international conference on Machine learning*, pages 697–704, 2006.
- [28] Kate Rakelly, Aurick Zhou, Chelsea Finn, Sergey Levine, and Deirdre Quillen. Efficient off-policy meta-reinforcement learning via probabilistic context variables. In *International conference on machine learning*, pages 5331–5340. PMLR, 2019.
- [29] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [30] Gresa Shala, André Biedenkapp, Pierre Krack, Florian Walter, and Josif Grabocka. Efficient cross-episode meta-rl. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [31] Carl-Johann Simon-Gabriel and Bernhard Schölkopf. Kernel distribution embeddings: Universal kernels, characteristic kernels and kernel metrics on distributions. *Journal of Machine Learning Research*, 19(44):1–29, 2018.
- [32] Bharath K Sriperumbudur, Kenji Fukumizu, and Gert RG Lanckriet. Universality, characteristic kernels and rkhs embedding of measures. *Journal of Machine Learning Research*, 12(7), 2011.
- [33] Malcolm Strens. A bayesian framework for reinforcement learning. In *ICML*, volume 2000, pages 943–950, 2000.
- [34] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 5026–5033. IEEE, 2012.
- [35] Nikolaos Tziortziotis, Christos Dimitrakakis, and Konstantinos Blekas. Linear bayesian reinforcement learning. In *IJCAI*, pages 1721–1728, 2013.

- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [37] Jane X Wang, Zeb Kurth-Nelson, Dhruva Tirumala, Hubert Soyer, Joel Z Leibo, Remi Munos, Charles Blundell, Dharshan Kumaran, and Matt Botvinick. Learning to reinforcement learn. *arXiv preprint arXiv:1611.05763*, 2016.
- [38] Jiachen Yang, Brenden Petersen, Hongyuan Zha, and Daniel Faissol. Single episode policy transfer in reinforcement learning. *arXiv preprint arXiv:1910.07719*, 2019.
- [39] Jiayu Yao, Taylor Killian, George Konidaris, and Finale Doshi-Velez. Direct policy transfer via hidden parameter markov decision processes. In *LLARLA Workshop, FAIM*, volume 2018, 2018.
- [40] Jaesik Yoon, Taesup Kim, Ousmane Dia, Sungwoong Kim, Yoshua Bengio, and Sungjin Ahn. Bayesian model-agnostic meta-learning. *Advances in neural information processing systems*, 31, 2018.
- [41] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Avnish Narayan, Hayden Shively, Adithya Bellathur, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning, 2021.
- [42] Luisa Zintgraf, Sebastian Schulze, Cong Lu, Leo Feng, Maximilian Igl, Kyriacos Shiarlis, Yarin Gal, Katja Hofmann, and Shimon Whiteson. Varibad: Variational bayes-adaptive deep rl via meta-learning. *Journal of Machine Learning Research*, 22(289):1–39, 2021.

A Appendix

A.1 Table of Important Notations

Symbol	Meaning
<i>MDPs and BAMDPs</i>	
$\mathcal{M} = (S, A, R, T, \gamma)$	MDP with state space S , action space A , reward R , transition T , discount γ .
$S^+ = S \times \mathcal{B}_T \times \mathcal{B}_R$	BAMDP hyper-state space; $\mathcal{B}_T, \mathcal{B}_R$ are the belief spaces of θ_T, θ_R .
$b_t = p(\theta_T, \theta_R)$	Belief at time t over task parameters.
τ	Belief-update function, $b_{t+1} = \tau(b_t, s_t, a_t, s_{t+1}, r_{t+1})$.
$\pi_\psi^+(a s, b)$	BAMDP (BRL) policy parameterised by ψ .
$H^+, \mathcal{J}^+(\pi^+)$	BAMDP horizon and expected return.
<i>Datasets and dimensions</i>	
$c_t = (s_t, a_t, s_{t+1}, r_{t+1})$	Single sample at timestep t .
$\mathcal{C}^{\mathcal{M}} = \{c_t\}_{t=1}^N$	N samples collected within MDP $\mathcal{M}$.
$\mathcal{C} = \bigcup_{i=1}^M \mathcal{C}^{\mathcal{M}_i}$	Joint dataset over a batch of M MDPs.
$\mathcal{C}^{\mathcal{M}_i} = (\mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i, \mathbf{r}_i)$	Samples in compact matrix forms.
D_S, D_A	State and action dimensions.
D_T, D_R	Transition and reward task dimensions.
D_P	Task-representation dimension.
<i>Task parameters</i>	
$\theta_T = (T_\mu, T_\sigma)$	Transition task parameter: mean T_μ , noise covariance T_σ .
$\theta_R = (R_\mu, R_\sigma)$	Reward task parameter: mean R_μ , noise covariance R_σ .
<i>Learnable basis functions</i>	
ϕ_T, ϕ_R	Neural networks acting as basis functions for transition and reward.
$\mathbf{C}_T = \phi_T(S_i, A_i)$	Transition features (design matrix).
$\mathbf{C}_R = \phi_R(S_i, A_i, S'_i)$	Reward features (design matrix).
ζ	Optional network parameterising the prior $p_\zeta(\theta_T, \theta_R)$.
<i>Distributions</i>	
$\mathcal{MN}(\mathbf{W} \mathbf{X}, \mathbf{Y}, \mathbf{Z})$	Matrix-normal: mean $\mathbf{X}$, row covariance $\mathbf{Y}$, column covariance $\mathbf{Z}$.
$\mathcal{W}(\mathbf{W} \mathbf{X}, \nu)$	Wishart: scale matrix $\mathbf{X}$, degrees of freedom ν .
<i>Prior/posterior parameters</i>	
$\mathbf{M}_T / \mathbf{M}'_T$	Prior/posterior mean of T_μ .
$\mathbf{\Xi}_T^{-1} / \mathbf{\Xi}'_T{}^{-1}$	Prior/posterior row covariance of T_μ ($\mathbf{\Xi}_T$ is the row precision).
$\mathbf{\Omega}_T / \mathbf{\Omega}'_T, \nu_T / \nu'_T$	Prior/posterior Wishart scale and dof on T_σ^{-1} . Parameters for θ_R follow analogously.
<i>Kernel-correspondence</i>	
k_{TT}, k_R, k_{TTR}	Kernel functions induced by the learnt basis (defined in Appendix A.9).
w_{TT}, w_R, w_{TTR}	Associated weighting functions.
$\text{sCMS}_k^w(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2})$	Empirical signed-measure Cosine Mean Similarity.
$\text{sMMD}_k^w(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2})$	Empirical signed-measure Maximum Mean Discrepancy.

A.2 GLiBRL Algorithm

Algorithm 1: GLiBRL

```

Initialise: policy  $\pi_\psi^+$ , horizon  $H$ ,  $\phi_T, \phi_R$ ;
while Training do
    Sample  $K$  MDPs  $\{\mathcal{M}_i\}_{i=1}^K$ ;
    Initialise: Joint data  $\mathcal{C} = \{\}$ ;
    // collecting samples
    for  $i \in \{1, 2, \dots, K\}$  do
        Initialise: samples in  $\mathcal{M}_i$ ,  $\mathcal{C}^{\mathcal{M}_i} = \{\}$ , state  $s$ ;
        for  $t < H$  do
             $b \leftarrow p_{\phi_T, \phi_R}(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i})$ ;
             $a \sim \pi_\psi^+(a | s, b)$ ;
            Execute  $a$  from  $s$  in  $\mathcal{M}_i$  to get  $s', r$ ;
             $\mathcal{C}^{\mathcal{M}_i} \leftarrow \mathcal{C}^{\mathcal{M}_i} \cup \{s, a, s', r\}$ ;
             $s \leftarrow s'$ ;
         $\mathcal{C} = \mathcal{C} \cup \mathcal{C}^{\mathcal{M}_i}$ 
    // learning policy, transition and reward models
    while Learning do
        Sample  $\mathcal{D} \subseteq \mathcal{C}$ ;
         $\psi \leftarrow \psi - \nabla_\psi \mathcal{L}_{\text{policy}}(\mathcal{D})$ ;
         $\phi_T \leftarrow \phi_T - \nabla_{\phi_T} \mathcal{L}_{\text{model}}(\mathcal{D})$ ;
         $\phi_R \leftarrow \phi_R - \nabla_{\phi_R} \mathcal{L}_{\text{model}}(\mathcal{D})$ ;

```

A.3 GLiBRL-SAC

Algorithm 2: GLiBRL with SAC

```

while Training do
    // collecting samples following Algorithm 1
    while Learning do
        Sample  $\mathcal{D} \subseteq \mathcal{C}$  of variable-length  $N \in [1, \text{Horizon}]$ ;
         $b \leftarrow p_{\phi_T, \phi_R}(\theta_T, \theta_R | \mathcal{D})$ ;
         $\psi \leftarrow \psi - \nabla_\psi (\mathcal{L}_{\text{actor}}(\mathcal{D}, b) + \mathcal{L}_{\text{critic}}(\mathcal{D}, b) + \mathcal{L}_\alpha)$ ;
        Polyak averaging for target Q-networks within  $\psi$ ;
        // Model updates following Algorithm 1

```

A.4 GLiBRL-PPO

Algorithm 3: GLiBRL with PPO

```

while Training do
    // collecting samples following Algorithm 1
    Compute advantages  $\hat{A}$  and return targets using  $\mathcal{C}$ ;
    while Learning do
        Sample  $\mathcal{D} \subseteq \mathcal{C}$  of fixed length  $N$ ;
         $\{b_i\}_{i=1}^N \leftarrow \{p_{\phi_T, \phi_R}(\theta_T, \theta_R | \mathcal{D}[1 : i])\}_{i=1}^N$ ;
         $\psi \leftarrow \psi - \nabla_\psi \mathcal{L}_{\text{actor}}(\mathcal{D}, \hat{A}, \{b_i\}_{i=1}^N)$ ;
        // Model updates following Algorithm 1

```

A.5 Evidence Lower Bound

The evidence lower bound in Equation 4 is derived as follows

$$\begin{aligned}
\log p_{\phi_T, \phi_R}(\mathcal{C}) &= \sum_{i=1}^M \log \iint p(\mathcal{C}^{\mathcal{M}_i}, \theta_T, \theta_R) \frac{q(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i})}{q(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i})} d\theta_T d\theta_R \\
&\geq \sum_{i=1}^M \mathbb{E}_q \left[\log \frac{p(\theta_T, \theta_R)}{q(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i})} + \log p(\mathcal{C}^{\mathcal{M}_i} | \theta_T, \theta_R) \right] \\
&= \sum_{i=1}^M \mathbb{E}_q [\log p(\mathcal{C}^{\mathcal{M}_i} | \theta_T, \theta_R)] - D_{KL}(q(\theta_T, \theta_R | \mathcal{C}^{\mathcal{M}_i}) \| p(\theta_T, \theta_R))
\end{aligned}$$

A.6 Normal-Wishart-Normal Conjugacy

Given Equation 6

$$p(\theta_T) = \mathcal{MN}(T_\mu | \mathbf{M}_T, \mathbf{\Xi}_T^{-1}, T_\sigma) \cdot \mathcal{W}(T_\sigma^{-1} | \mathbf{\Omega}_T^{-1}, \nu_T)$$

and Equation 9

$$p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i, \theta_T) = \mathcal{MN}(\mathbf{S}'_i | \mathbf{C}_T T_\mu, \mathbf{I}_N, T_\sigma) \quad (9)$$

We prove Equation 13

$$p_{\phi_T}(\theta_T | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i) = \mathcal{MN}(T_\mu | \mathbf{M}'_T, \mathbf{\Xi}'_T^{-1}, T_\sigma) \cdot \mathcal{W}(T_\sigma^{-1} | \mathbf{\Omega}'_T^{-1}, \nu'_T) \quad (13)$$

where

$$\begin{aligned}
\mathbf{M}'_T &= \mathbf{\Xi}'_T^{-1} [\mathbf{C}_T^T \mathbf{S}' + \mathbf{\Xi}_T \mathbf{M}_T] \\
\mathbf{\Xi}'_T &= \mathbf{C}_T^T \mathbf{C}_T + \mathbf{\Xi}_T \\
\mathbf{\Omega}'_T &= \mathbf{\Omega}_T + \mathbf{S}'^T \mathbf{S}' \\
&\quad + \mathbf{M}_T^T \mathbf{\Xi}_T \mathbf{M}_T - \mathbf{M}_T^T \mathbf{\Xi}'_T \mathbf{M}'_T \\
\nu'_T &= \nu_T + N
\end{aligned} \quad (15)$$

Proof:

The density of the prior distribution $p(\theta_T)$ is

$$p(\theta_T) = \frac{|\mathbf{\Xi}_T|^{D_S/2}}{\sqrt{(2\pi)^{D_T D_S}} |T_\sigma|^{D_T/2}} \cdot \exp \left[-\frac{1}{2} \text{Tr} (T_\sigma^{-1} (T_\mu - \mathbf{M}_T)^T \mathbf{\Xi}_T (T_\mu - \mathbf{M}_T)) \right] \quad (25)$$

$$\begin{aligned}
&\cdot \sqrt{\frac{|\mathbf{\Omega}_T|^{\nu_T}}{2^{\nu_T D_S}}} \frac{|T_\sigma|^{(1+D_S-\nu_T)/2}}{\Gamma_{D_S}(\nu_T/2)} \cdot \exp \left[-\frac{1}{2} \text{Tr} (\mathbf{\Omega}_T T_\sigma^{-1}) \right] \\
&\propto |T_\sigma|^{(1+D_S-\nu_T-D_T)/2} \cdot \exp \left\{ -\frac{1}{2} \text{Tr} \left[T_\sigma^{-1} \left[(T_\mu - \mathbf{M}_T)^T \mathbf{\Xi}_T (T_\mu - \mathbf{M}_T) + \mathbf{\Omega}_T \right] \right] \right\} \quad (26)
\end{aligned}$$

where from Equation 25 to Equation 26 we treat multiplicative parameters irrelevant to θ_T as constants.

The joint density of $p_{\phi_T}(\theta_T, \mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i) = p(\theta_T) \cdot p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i, \theta_T)$ is hence

$$\begin{aligned}
&\frac{|\mathbf{\Xi}_T|^{D_S/2}}{\sqrt{(2\pi)^{D_T D_S}} |T_\sigma|^{D_T/2}} \cdot \exp \left[-\frac{1}{2} \text{Tr} (T_\sigma^{-1} (T_\mu - \mathbf{M}_T)^T \mathbf{\Xi}_T (T_\mu - \mathbf{M}_T)) \right] \\
&\cdot \sqrt{\frac{|\mathbf{\Omega}_T|^{\nu_T}}{2^{\nu_T D_S}}} \frac{|T_\sigma|^{(1+D_S-\nu_T)/2}}{\Gamma_{D_S}(\nu_T/2)} \cdot \exp \left[-\frac{1}{2} \text{Tr} (\mathbf{\Omega}_T T_\sigma^{-1}) \right] \quad (27)
\end{aligned}$$

$$\begin{aligned}
&\cdot \frac{1}{\sqrt{(2\pi)^{N D_S}} |T_\sigma|^{N/2}} \cdot \exp \left[-\frac{1}{2} \text{Tr} (T_\sigma^{-1} (\mathbf{S}'_i - \mathbf{C}_T T_\mu)^T (\mathbf{S}'_i - \mathbf{C}_T T_\mu)) \right] \\
&\propto |T_\sigma|^{(1+D_S-\nu_T-N-D_T)/2} \cdot \exp \left[-\frac{1}{2} \text{Tr} (T_\sigma^{-1} (T_\mu - \mathbf{M}_T)^T \mathbf{\Xi}_T (T_\mu - \mathbf{M}_T)) \right] \\
&\quad \cdot \exp \left[-\frac{1}{2} \text{Tr} (T_\sigma^{-1} ((\mathbf{S}'_i - \mathbf{C}_T T_\mu)^T (\mathbf{S}'_i - \mathbf{C}_T T_\mu) + \mathbf{\Omega}_T)) \right] \quad (28)
\end{aligned}$$

Matching the second-order then the first-order term with respect to T_μ , we can rewrite

$$28 = |T_\sigma|^{(1+D_S-\nu'_T-D_T)/2} \cdot \exp \left\{ -\frac{1}{2} \text{Tr} \left[T_\sigma^{-1} \left[(T_\mu - \mathbf{M}'_T)^T \Xi'_T (T_\mu - \mathbf{M}'_T) + \Omega'_T \right] \right] \right\} \quad (29)$$

We can find Equation 26 and Equation 29 match exactly, indicating the Normal-Wishart-Normal conjugacy. Note, we just use the posterior update of $p(\theta_T)$ as an example. The exact same proof applies to the posterior update of $p(\theta_R)$ as well. Such conjugacy allows exact posterior update and marginal likelihood, enabling efficient learning.

A.7 Marginal Log-likelihood of Normal-Wishart-Normal

We prove Equation 19 has the following closed form

$$\log p_{\phi_T, \phi_R}(\mathcal{C}) = -\frac{1}{2} \sum_{i=1}^M \left(D_S \log |\Xi'_T| + \nu'_T \log \left| \frac{1}{2} \Omega'_T \right| + \log |\Xi'_R| + \nu'_R \log \left| \frac{1}{2} \Omega'_R \right| \right) + \text{const.} \quad (19)$$

Proof:

Consider

$$p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i) = \frac{p_{\phi_T}(\theta_T, \mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i)}{p_{\phi_T}(\theta_T | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i)} \quad (30)$$

From Appendix A.6, we know the numerator is

$$\frac{|\Xi_T|^{D_S/2} |\Omega_T|^{\nu_T/2}}{2^{\nu_T D_S/2} \cdot (2\pi)^{D_S(D_T+N)/2} \cdot \Gamma_{D_S}(\nu_T/2)} \cdot \text{Equation 29} \quad (31)$$

The denominator is

$$\frac{|\Xi'_T|^{D_S/2} |\Omega'_T|^{\nu'_T/2}}{2^{\nu'_T D_S/2} \cdot (2\pi)^{D_S D_T/2} \cdot \Gamma_{D_S}(\nu'_T/2)} \cdot \text{Equation 29} \quad (32)$$

Hence,

$$p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i) = \frac{1}{(2\pi)^{D_S N/2}} \cdot \frac{|\Xi_T|^{D_S/2} |\frac{1}{2} \Omega_T|^{\nu_T/2} \cdot \Gamma_{D_S}(\nu'_T/2)}{|\Xi'_T|^{D_S/2} |\frac{1}{2} \Omega'_T|^{\nu'_T/2} \cdot \Gamma_{D_S}(\nu_T/2)} \quad (33)$$

Similarly,

$$p_{\phi_R}(\mathbf{r}_i | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i) = \frac{1}{(2\pi)^{N/2}} \cdot \frac{|\Xi_R|^{1/2} |\frac{1}{2} \Omega_R|^{\nu_R/2} \cdot \Gamma(\nu'_R/2)}{|\Xi'_R|^{1/2} |\frac{1}{2} \Omega'_R|^{\nu_R/2} \cdot \Gamma(\nu_R/2)} \quad (34)$$

Note that

$$p_{\phi_T, \phi_R}(\mathcal{C}^{\mathcal{M}_i}) = p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i) p_{\phi_R}(\mathbf{r}_i | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i) p(\mathbf{S}_i, \mathbf{A}_i) \quad (35)$$

By taking the logarithm, and note the independence of $p(\mathbf{S}_i, \mathbf{A}_i)$ with respect to ϕ_T, ϕ_R ,

$$\log p_{\phi_T, \phi_R}(\mathcal{C}^{\mathcal{M}_i}) = \log p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i) + \log p_{\phi_R}(\mathbf{r}_i | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i) + \text{const.} \quad (36)$$

Sum the above equation on both sides with respect to i , then we have Equation 19.

A.8 Choices of Belief Representations

We list several candidate f s:

$$\begin{aligned} f_1(\mathbf{M}_T, \mathbf{M}_R) &= [\text{vec}(\mathbf{M}_T)^T \mathbf{M}_R^T]^T & f_2(\mathbf{M}_T, \mathbf{M}_R) &= [\text{tril}(\mathbf{M}_T \mathbf{M}_T^T)^T \mathbf{M}_R^T]^T \\ f_3(\mathbf{M}_T, \mathbf{M}_R) &= [\text{tril}(\mathbf{M}_T^T \mathbf{M}_T)^T \mathbf{M}_R^T]^T & f_4(\mathbf{M}_T, \mathbf{M}_R) &= \left[\frac{\text{tril}(\mathbf{M}_T \mathbf{M}_T^T)}{\|\text{tril}(\mathbf{M}_T \mathbf{M}_T^T)\|_2}^T \frac{\mathbf{M}_R^T}{\|\mathbf{M}_R\|_2} \right]^T \\ f_5(\mathbf{M}_T, \mathbf{M}_R) &= \left[\frac{\text{tril}(\mathbf{M}_T^T \mathbf{M}_T)}{\|\text{tril}(\mathbf{M}_T^T \mathbf{M}_T)\|_2}^T \frac{\mathbf{M}_R^T}{\|\mathbf{M}_R\|_2} \right]^T & f_6(\mathbf{M}_T, \mathbf{M}_R) &= [g_T(\mathbf{M}_T)^T g_R(\mathbf{M}_R)]^T \end{aligned}$$

where $\text{tril}(\cdot)$ retrieves the flattened lower triangle of matrices and $g_T(\cdot), g_R(\cdot)$ are neural networks. f_1 directly flattens and concatenates the task means, resulting in $D_T \times D_S + D_R$ dimensional representations. f_2 and f_3 are used for scenarios with large D_S and D_T , giving $D_T(D_T+1)/2 + D_R$ and $D_S(D_S+1)/2 + D_R$ dimensional representations respectively. f_4 and f_5 normalise the representations. f_6 suggests learnable belief representations.

f_1 to f_5 can all be shown to be equivalent to the kernalised empirical correspondence of task samples from proofs in Appendix A.9. Because of the use of neural networks in f_6 , it is difficult to show similar correspondence in general. Neural networks in f_6 are trained by backproagating the Reinforcement Learning loss. Empirically, we have observed it performing worse than f_4 , or $f_{T,R}$ defined in Section 4.4.

A.9 Equivalence of Bayesian Parameter Shift, sMMD and sCMS

Consider two arbitrary sets of samples, which come from two tasks $\mathcal{M}_1$ and $\mathcal{M}_2$:

$$\begin{aligned}\mathcal{C}^{\mathcal{M}_1} &= \{s_{1i}, a_{1i}, s_{1i+1}, r_{1i+1}\}_{i=1}^M = (\mathbf{S}_1, \mathbf{A}_1, \mathbf{S}'_1, \mathbf{r}_1) \\ \mathcal{C}^{\mathcal{M}_2} &= \{s_{2i}, a_{2i}, s_{2i+1}, r_{2i+1}\}_{i=1}^N = (\mathbf{S}_2, \mathbf{A}_2, \mathbf{S}'_2, \mathbf{r}_2)\end{aligned}$$

and corresponding features

$$\mathbf{C}_T^1 = \phi_T(\mathbf{S}_1, \mathbf{A}_1) \quad \mathbf{C}_R^1 = \phi_R(\mathbf{S}_1, \mathbf{A}_1, \mathbf{S}'_1) \quad \mathbf{C}_T^2 = \phi_T(\mathbf{S}_2, \mathbf{A}_2) \quad \mathbf{C}_R^2 = \phi_R(\mathbf{S}_2, \mathbf{A}_2, \mathbf{S}'_2)$$

Define the GLiBRL task posterior means $\mathbf{M}_T^{1'}, \mathbf{M}_R^{1'}, \mathbf{M}_T^{2'}, \mathbf{M}_R^{2'}$, assuming shared prior means $\mathbf{M}_T = \mathbf{0}, \mathbf{M}_R = \mathbf{0}$ and covariances $\mathbf{\Xi}_T, \mathbf{\Xi}_R$, where the posterior update follows Equation 15. Note that the zero prior mean is only assumed for notational conciseness but not necessary. For example, let $\mathbf{C}_T^T \mathbf{S}' + \mathbf{\Xi}_T \mathbf{M}_T = [\mathbf{C}_T^T \quad \mathbf{I}] [\mathbf{S}'^T \quad \mathbf{M}_T^T \mathbf{\Xi}_T^T]^T = \widehat{\mathbf{C}}_T^T \widehat{\mathbf{S}}'$. One can proceed with $\widehat{\mathbf{C}}_T$ and $\widehat{\mathbf{S}}'$ following similar proofs to be introduced shortly.

Define functions $w(\cdot)$ mapping from $\mathcal{C}^{\mathcal{M}_1}$ and $\mathcal{C}^{\mathcal{M}_2}$ to vectors. We define the empirical signed-measure Maximum-Mean Discrepancy (sMMD) between $\mathcal{C}^{\mathcal{M}_1}$ and $\mathcal{C}^{\mathcal{M}_2}$ under kernel function $k(\cdot, \cdot)$ and signed weighting function $w(\cdot)$ as

$$\begin{aligned}\text{sMMD}_k^w(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) &= w(\mathcal{C}^{\mathcal{M}_1})^T \mathbf{K}_{11} w(\mathcal{C}^{\mathcal{M}_1}) + w(\mathcal{C}^{\mathcal{M}_2})^T \mathbf{K}_{22} w(\mathcal{C}^{\mathcal{M}_2}) \\ &\quad - 2w(\mathcal{C}^{\mathcal{M}_1})^T \mathbf{K}_{12} w(\mathcal{C}^{\mathcal{M}_2})\end{aligned}\quad (37)$$

where entry at row m and column n of matrix $\mathbf{K}_{ij}[m, n]$ equals the kernel function evaluated with the row m of $\mathcal{C}^{\mathcal{M}_i}$ and the row n of $\mathcal{C}^{\mathcal{M}_j}$, i.e. $\mathbf{K}_{ij}[m, n] = k(\mathcal{C}^{\mathcal{M}_i}[m], \mathcal{C}^{\mathcal{M}_j}[n])$. Similarly, we define the empirical signed-measure Cosine Mean Similarity (sCMS) between $\mathcal{C}^{\mathcal{M}_1}$ and $\mathcal{C}^{\mathcal{M}_2}$ under kernel function $k(\cdot, \cdot)$ and signed weighting function $w(\cdot)$ as

$$\text{sCMS}_k^w(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) = \frac{w(\mathcal{C}^{\mathcal{M}_1})^T \mathbf{K}_{12} w(\mathcal{C}^{\mathcal{M}_2})}{\sqrt{w(\mathcal{C}^{\mathcal{M}_1})^T \mathbf{K}_{11} w(\mathcal{C}^{\mathcal{M}_1})} \cdot \sqrt{w(\mathcal{C}^{\mathcal{M}_2})^T \mathbf{K}_{22} w(\mathcal{C}^{\mathcal{M}_2})}}\quad (38)$$

sMMD and sCMS extend the traditional MMD [12] and CMS [14] defined on probability measures to signed measures covered by the general Reproducing Kernel Hilbert Space (RKHS) theory [32, 31] to accommodate the general RL setting where states and rewards can take negatives values and unnormalised.

We claim that there exists kernel functions $k(\cdot, \cdot)$ s and signed weighting functions $w(\cdot)$ s, such that the following statements hold:

$$\left\| \mathbf{M}_R^{1'} - \mathbf{M}_R^{2'} \right\|_2^2 = \text{sMMD}_{k_R}^{w_R} (\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) \quad (39)$$

$$\left\| \mathbf{M}_T^{1'} - \mathbf{M}_T^{2'} \right\|_F^2 = \text{sMMD}_{k_T}^{w_T} (\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) \quad (40)$$

$$\left\| \text{tril}(\mathbf{M}_T^{1'} \mathbf{M}_T^{1'T}) - \text{tril}(\mathbf{M}_T^{2'} \mathbf{M}_T^{2'T}) \right\|_2^2 = \text{sMMD}_{k_{TT}}^{w_{TT}} (\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) \quad (41)$$

$$\left\| \frac{\mathbf{M}_R^{1'}}{\left\| \mathbf{M}_R^{1'} \right\|_2} - \frac{\mathbf{M}_R^{2'}}{\left\| \mathbf{M}_R^{2'} \right\|_2} \right\|_2^2 = 2 \cdot (1 - \text{sCMS}_{k_R}^{w_R} (\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2})) \quad (42)$$

$$\left\| \frac{\text{tril}(\mathbf{M}_T^{1'} \mathbf{M}_T^{1'T})}{\left\| \text{tril}(\mathbf{M}_T^{1'} \mathbf{M}_T^{1'T}) \right\|_2} - \frac{\text{tril}(\mathbf{M}_T^{2'} \mathbf{M}_T^{2'T})}{\left\| \text{tril}(\mathbf{M}_T^{2'} \mathbf{M}_T^{2'T}) \right\|_2} \right\|_2^2 = 2 \cdot (1 - \text{sCMS}_{k_{TT}}^{w_{TT}} (\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2})) \quad (43)$$

$$\begin{aligned} & \left\| f_{T,R}(\mathbf{M}_T^{1'}, \mathbf{M}_R^{1'}) - f_{T,R}(\mathbf{M}_T^{2'}, \mathbf{M}_R^{2'}) \right\|_2^2 \\ &= 4 - 2 \cdot (\text{sCMS}_{k_{TT}}^{w_{TT}} (\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) + \text{sCMS}_{k_R}^{w_R} (\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2})) \end{aligned} \quad (23)$$

$$\left\| \widetilde{f_{T,R}}(\mathbf{M}_T^{1'}, \mathbf{M}_R^{1'}) - \widetilde{f_{T,R}}(\mathbf{M}_T^{2'}, \mathbf{M}_R^{2'}) \right\|_2^2 = \text{sMMD}_{k_{TT}R}^{w_{TT}R} (\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) \quad (24)$$

where $f_{T,R}$ and $\widetilde{f_{T,R}}$ are defined in Section 4.4.

Proof of Equation 40:

From Equation 15,

$$\mathbf{M}_T^{1'} = (\mathbf{C}_T^1 \mathbf{C}_T^1 + \mathbf{\Xi}_T)^{-1} \mathbf{C}_T^1 \mathbf{S}'_1 \quad (44)$$

$$\mathbf{M}_T^{2'} = (\mathbf{C}_T^2 \mathbf{C}_T^2 + \mathbf{\Xi}_T)^{-1} \mathbf{C}_T^2 \mathbf{S}'_2 \quad (45)$$

Let

$$\widetilde{\mathbf{S}}_1 = \mathbf{S}'_1 \quad \mathbf{L}_1^T = (\mathbf{C}_T^1 \mathbf{C}_T^1 + \mathbf{\Xi}_T)^{-1} \mathbf{C}_T^1 \mathbf{T} \quad (46)$$

$$\widetilde{\mathbf{S}}_2 = \mathbf{S}'_2 \quad \mathbf{L}_2^T = (\mathbf{C}_T^2 \mathbf{C}_T^2 + \mathbf{\Xi}_T)^{-1} \mathbf{C}_T^2 \mathbf{T} \quad (47)$$

Therefore, the squared Frobenius norm of $\mathbf{M}_T^{1'} - \mathbf{M}_T^{2'}$ is

$$\begin{aligned} \left\| \mathbf{M}_T^{1'} - \mathbf{M}_T^{2'} \right\|_F^2 &= \text{Tr} \left[(\mathbf{M}_T^{1'} - \mathbf{M}_T^{2'})^T (\mathbf{M}_T^{1'} - \mathbf{M}_T^{2'}) \right] \\ &= \text{Tr} \left[\widetilde{\mathbf{S}}_1^T \mathbf{L}_1 \mathbf{L}_1^T \widetilde{\mathbf{S}}_1 \right] + \text{Tr} \left[\widetilde{\mathbf{S}}_2^T \mathbf{L}_2 \mathbf{L}_2^T \widetilde{\mathbf{S}}_2 \right] - 2 \text{Tr} \left[\widetilde{\mathbf{S}}_1^T \mathbf{L}_1 \mathbf{L}_2^T \widetilde{\mathbf{S}}_2 \right] \\ &= \text{vec}(\widetilde{\mathbf{S}}_1)^T ((\mathbf{I} \otimes \mathbf{L}_1) (\mathbf{I} \otimes \mathbf{L}_1)^T) \text{vec}(\widetilde{\mathbf{S}}_1) \\ &\quad + \text{vec}(\widetilde{\mathbf{S}}_2)^T ((\mathbf{I} \otimes \mathbf{L}_2) (\mathbf{I} \otimes \mathbf{L}_2)^T) \text{vec}(\widetilde{\mathbf{S}}_2) \\ &\quad - 2 \cdot \text{vec}(\widetilde{\mathbf{S}}_1)^T ((\mathbf{I} \otimes \mathbf{L}_1) (\mathbf{I} \otimes \mathbf{L}_2)^T) \text{vec}(\widetilde{\mathbf{S}}_2) \end{aligned} \quad (48)$$

Let $\mathbf{B}_1 = \mathbf{I} \otimes \mathbf{L}_1$ and $\mathbf{B}_2 = \mathbf{I} \otimes \mathbf{L}_2$. Equation 40 is equivalent to

$$\begin{aligned} \left\| \mathbf{M}_T^{1'} - \mathbf{M}_T^{2'} \right\|_F^2 &= \text{vec}(\widetilde{\mathbf{S}}_1)^T \mathbf{B}_1 \mathbf{B}_1^T \text{vec}(\widetilde{\mathbf{S}}_1) + \text{vec}(\widetilde{\mathbf{S}}_2)^T \mathbf{B}_2 \mathbf{B}_2^T \text{vec}(\widetilde{\mathbf{S}}_2) \\ &\quad - 2 \cdot \text{vec}(\widetilde{\mathbf{S}}_1)^T \mathbf{B}_1 \mathbf{B}_2^T \text{vec}(\widetilde{\mathbf{S}}_2) \end{aligned} \quad (49)$$

Thus, we have shown

$$\left\| \mathbf{M}_T^{1'} - \mathbf{M}_T^{2'} \right\|_F^2 = \text{sMMD}_{k_T}^{w_T} (\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) \quad (40)$$

where

$$w_T(\mathcal{C}^{\mathcal{M}_i}) = \text{vec}(\widetilde{\mathbf{S}}_i) \quad k_T(\mathcal{C}^{\mathcal{M}_i}[m], \mathcal{C}^{\mathcal{M}_j}[n]) = \mathbf{B}_i[m]^T \mathbf{B}_j[n]$$

Proof of Equation 41:

Define a matrix $\mathbf{D}$ whose entries are zeros or ones that only depends on the value of D_T , and $\mathbf{D} \text{vec}(\mathbf{M}_T^1 \mathbf{M}_T^{1'} - \mathbf{M}_T^2 \mathbf{M}_T^{2'}) = \text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'}) - \text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'})$. Let

$$\widetilde{\mathbf{S}\mathbf{S}}_1 = \mathbf{S}_1' \mathbf{S}_1^T \quad \mathbf{L}_1^T = (\mathbf{C}_T^1 \mathbf{C}_T^1 + \mathbf{\Xi}_T)^{-1} \mathbf{C}_T^1 \mathbf{T} \quad (50)$$

$$\widetilde{\mathbf{S}\mathbf{S}}_2 = \mathbf{S}_2' \mathbf{S}_2^T \quad \mathbf{L}_2^T = (\mathbf{C}_T^2 \mathbf{C}_T^2 + \mathbf{\Xi}_T)^{-1} \mathbf{C}_T^2 \mathbf{T} \quad (51)$$

Equation 41 is hence equivalent to

$$\begin{aligned} \left\| \text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'}) - \text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'}) \right\|_2^2 &= \text{vec}(\mathbf{L}_1^T \mathbf{S}_1' \mathbf{S}_1^T \mathbf{L}_1)^T \mathbf{D}^T \mathbf{D} \text{vec}(\mathbf{L}_1^T \mathbf{S}_1' \mathbf{S}_1^T \mathbf{L}_1) + \\ &\text{vec}(\mathbf{L}_2^T \mathbf{S}_2' \mathbf{S}_2^T \mathbf{L}_2)^T \mathbf{D}^T \mathbf{D} \text{vec}(\mathbf{L}_2^T \mathbf{S}_2' \mathbf{S}_2^T \mathbf{L}_2) - 2 \cdot \text{vec}(\mathbf{L}_1^T \mathbf{S}_1' \mathbf{S}_1^T \mathbf{L}_1)^T \mathbf{D}^T \mathbf{D} \text{vec}(\mathbf{L}_2^T \mathbf{S}_2' \mathbf{S}_2^T \mathbf{L}_2) \end{aligned} \quad (52)$$

Apply $\text{vec}(\mathbf{U}\mathbf{V}\mathbf{W}) = (\mathbf{W}^T \otimes \mathbf{U}) \text{vec}(\mathbf{V})$, and let

$$\mathbf{B}_1 = (\mathbf{L}_1 \otimes \mathbf{L}_1) \mathbf{D}^T \quad \mathbf{B}_2 = (\mathbf{L}_2 \otimes \mathbf{L}_2) \mathbf{D}^T \quad (53)$$

Equation 41 reduces to

$$\begin{aligned} \left\| \text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'}) - \text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'}) \right\|_2^2 &= \text{vec}(\widetilde{\mathbf{S}\mathbf{S}}_1)^T \mathbf{B}_1 \mathbf{B}_1^T \text{vec}(\widetilde{\mathbf{S}\mathbf{S}}_1) \\ &+ \text{vec}(\widetilde{\mathbf{S}\mathbf{S}}_2)^T \mathbf{B}_2 \mathbf{B}_2^T \text{vec}(\widetilde{\mathbf{S}\mathbf{S}}_2) - 2 \cdot \text{vec}(\widetilde{\mathbf{S}\mathbf{S}}_1)^T \mathbf{B}_1 \mathbf{B}_2^T \text{vec}(\widetilde{\mathbf{S}\mathbf{S}}_2) \end{aligned} \quad (54)$$

Hence, we have proved

$$\left\| \text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'}) - \text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'}) \right\|_2^2 = \text{sMMD}_{k_{TT}}^{w_{TT}}(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) \quad (41)$$

where

$$w_{TT}(\mathcal{C}^{\mathcal{M}_i}) = \text{vec}(\widetilde{\mathbf{S}\mathbf{S}}_i) \quad k_{TT}(\mathcal{C}^{\mathcal{M}_i}[m], \mathcal{C}^{\mathcal{M}_j}[n]) = \mathbf{B}_i[m]^T \mathbf{B}_j[n]$$

Proof of Equation 39:

This is a degenerate case of Equation 40 as $\mathbf{M}_R$ are vectors rather than matrices. Following a similar approach, and let

$$\widetilde{\mathbf{r}}_1 = \mathbf{r}_1 \quad \mathbf{L}_1^T = (\mathbf{C}_R^1 \mathbf{C}_R^1 + \mathbf{\Xi}_R)^{-1} \mathbf{C}_R^1 \mathbf{T} \quad (55)$$

$$\widetilde{\mathbf{r}}_2 = \mathbf{r}_2 \quad \mathbf{L}_2^T = (\mathbf{C}_R^2 \mathbf{C}_R^2 + \mathbf{\Xi}_R)^{-1} \mathbf{C}_R^2 \mathbf{T} \quad (56)$$

The squared $\mathcal{L}_2$ norm of $\mathbf{M}_R^2 - \mathbf{M}_R^1$ is

$$\left\| \mathbf{M}_R^2 - \mathbf{M}_R^1 \right\|_2^2 = (\mathbf{M}_R^2 - \mathbf{M}_R^1)^T (\mathbf{M}_R^2 - \mathbf{M}_R^1) \quad (57)$$

$$= \widetilde{\mathbf{r}}_1^T \mathbf{L}_1 \mathbf{L}_1^T \widetilde{\mathbf{r}}_1 + \widetilde{\mathbf{r}}_2^T \mathbf{L}_2 \mathbf{L}_2^T \widetilde{\mathbf{r}}_2 - 2 \cdot \widetilde{\mathbf{r}}_1^T \mathbf{L}_1 \mathbf{L}_2^T \widetilde{\mathbf{r}}_2 \quad (58)$$

Thus, we have shown

$$\left\| \mathbf{M}_R^1 - \mathbf{M}_R^2 \right\|_2^2 = \text{sMMD}_{k_R}^{w_R}(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) \quad (39)$$

where

$$w_R(\mathcal{C}^{\mathcal{M}_i}) = \widetilde{\mathbf{r}}_i \quad k_R(\mathcal{C}^{\mathcal{M}_i}[m], \mathcal{C}^{\mathcal{M}_j}[n]) = \mathbf{L}_i[m]^T \mathbf{L}_j[n]$$

Proof of Equation 43

Following the proof of Equation 41, Equation 43 is equivalent to

$$\left\| \frac{\text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'T})}{\|\text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'T})\|_2} - \frac{\text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'T})}{\|\text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'T})\|_2} \right\|_2^2 = 2 - 2 \cdot \frac{\text{vec}(\mathbf{M}_T^1 \mathbf{M}_T^{1'T})^T \mathbf{D}^T \mathbf{D} \text{vec}(\mathbf{M}_T^2 \mathbf{M}_T^{2'T})}{\|\text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'T})\|_2 \|\text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'T})\|_2} \quad (59)$$

Further reducing Equation 43 following the definition of $\mathbf{B}$ and $\widetilde{\mathbf{SS}}$ when proving Equation 41

$$2 - 2 \cdot \frac{\text{vec}(\widetilde{\mathbf{SS}}_1)^T \mathbf{B}_1 \mathbf{B}_2^T \text{vec}(\widetilde{\mathbf{SS}}_2)}{\sqrt{\text{vec}(\widetilde{\mathbf{SS}}_1)^T \mathbf{B}_1 \mathbf{B}_1^T \text{vec}(\widetilde{\mathbf{SS}}_1)} \cdot \sqrt{\text{vec}(\widetilde{\mathbf{SS}}_2)^T \mathbf{B}_2 \mathbf{B}_2^T \text{vec}(\widetilde{\mathbf{SS}}_2)}} \quad (60)$$

$$= 2 \cdot (1 - \text{sCMS}_{k_{TT}}^{w_{TT}}(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2})) \quad (61)$$

Proof of Equation 42

$$\left\| \frac{\mathbf{M}_R^1}{\|\mathbf{M}_R^1\|_2} - \frac{\mathbf{M}_R^2}{\|\mathbf{M}_R^2\|_2} \right\|_2^2 = 2 - 2 \cdot \frac{\mathbf{M}_R^1{}^T \mathbf{M}_R^2}{\|\mathbf{M}_R^1\|_2 \|\mathbf{M}_R^2\|_2} \quad (62)$$

Following the same definition of $\mathbf{L}$ and $\widetilde{\mathbf{r}}$ when proving Equation 39. Then,

$$\left\| \frac{\mathbf{M}_R^1}{\|\mathbf{M}_R^1\|_2} - \frac{\mathbf{M}_R^2}{\|\mathbf{M}_R^2\|_2} \right\|_2^2 = 2 - 2 \cdot \frac{\widetilde{\mathbf{r}}_1^T \mathbf{L}_1 \mathbf{L}_2^T \widetilde{\mathbf{r}}_2}{\sqrt{\widetilde{\mathbf{r}}_1^T \mathbf{L}_1 \mathbf{L}_1^T \widetilde{\mathbf{r}}_1} \cdot \sqrt{\widetilde{\mathbf{r}}_2^T \mathbf{L}_2 \mathbf{L}_2^T \widetilde{\mathbf{r}}_2}} \quad (63)$$

$$= 2 \cdot (1 - \text{sCMS}_{k_R}^{w_R}(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2})) \quad (64)$$

Proof of Equation 23

The proof is straightforward. Equation 23 is equivalent to

$$4 - 2 \cdot \left(\frac{\text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'T})^T \text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'T})}{\|\text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'T})\|_2 \|\text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'T})\|_2} + \frac{\mathbf{M}_R^1{}^T \mathbf{M}_R^2}{\|\mathbf{M}_R^1\|_2 \|\mathbf{M}_R^2\|_2} \right) \quad (65)$$

Using results from Equation 42 and Equation 43, we complete the proof immediately.

Proof of Equation 24

$\left\| \widetilde{f_{T,R}}(\mathbf{M}_T^1, \mathbf{M}_R^1) - \widetilde{f_{T,R}}(\mathbf{M}_T^2, \mathbf{M}_R^2) \right\|_2^2$ is equal to

$$\begin{aligned} & \text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'T})^T \text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'T}) + \text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'T})^T \text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'T}) \\ & + \mathbf{M}_R^1{}^T \mathbf{M}_R^1 + \mathbf{M}_R^2{}^T \mathbf{M}_R^2 - 2 \cdot \text{tril}(\mathbf{M}_T^1 \mathbf{M}_T^{1'T})^T \text{tril}(\mathbf{M}_T^2 \mathbf{M}_T^{2'T}) - 2 \cdot \mathbf{M}_R^1{}^T \mathbf{M}_R^2 \end{aligned} \quad (66)$$

For clarity, we define the design matrices $\mathbf{B}_1, \mathbf{B}_2$ in the proof of Equation 41 and Equation 39 as $\mathbf{B}_T^1, \mathbf{B}_T^2$ and $\mathbf{B}_R^1, \mathbf{B}_R^2$, respectively. Therefore, we obtain

$$\begin{aligned} & \text{vec}(\widetilde{\mathbf{SS}}_1)^T \mathbf{B}_T^1 \mathbf{B}_T^{1T} \text{vec}(\widetilde{\mathbf{SS}}_1) + \text{vec}(\widetilde{\mathbf{SS}}_2)^T \mathbf{B}_T^2 \mathbf{B}_T^{2T} \text{vec}(\widetilde{\mathbf{SS}}_2) + \widetilde{\mathbf{r}}_1^T \mathbf{B}_R^1 \mathbf{B}_R^{1T} \widetilde{\mathbf{r}}_1 \\ & + \widetilde{\mathbf{r}}_2^T \mathbf{B}_R^2 \mathbf{B}_R^{2T} \widetilde{\mathbf{r}}_2 - 2 \cdot \widetilde{\mathbf{r}}_1^T \mathbf{B}_R^1 \mathbf{B}_R^{2T} \widetilde{\mathbf{r}}_2 - 2 \cdot \text{vec}(\widetilde{\mathbf{SS}}_1)^T \mathbf{B}_T^1 \mathbf{B}_T^{2T} \text{vec}(\widetilde{\mathbf{SS}}_2) \end{aligned} \quad (67)$$

Let

$$\mathbf{sr}_1 = \begin{bmatrix} \text{vec}(\widetilde{\mathbf{SS}}_1)^\top & \widetilde{\mathbf{r}}_1^\top \end{bmatrix}^\top \quad \mathbf{sr}_2 = \begin{bmatrix} \text{vec}(\widetilde{\mathbf{SS}}_2)^\top & \widetilde{\mathbf{r}}_2^\top \end{bmatrix}^\top \quad (68)$$

$$\mathbf{K}_{11} = \begin{bmatrix} \mathbf{B}_T^1 \mathbf{B}_T^{1\top} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_R^1 \mathbf{B}_R^{1\top} \end{bmatrix} \quad \mathbf{K}_{22} = \begin{bmatrix} \mathbf{B}_T^2 \mathbf{B}_T^{2\top} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_R^2 \mathbf{B}_R^{2\top} \end{bmatrix} \quad \mathbf{K}_{12} = \begin{bmatrix} \mathbf{B}_T^1 \mathbf{B}_T^{2\top} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_R^1 \mathbf{B}_R^{2\top} \end{bmatrix} \quad (69)$$

Thus

$$\left\| \widetilde{f_{T,R}}(\mathbf{M}_T^{1'}, \mathbf{M}_R^{1'}) - \widetilde{f_{T,R}}(\mathbf{M}_T^{2'}, \mathbf{M}_R^{2'}) \right\|_2^2 = \mathbf{sr}_1^\top \mathbf{K}_{11} \mathbf{sr}_1 + \mathbf{sr}_2^\top \mathbf{K}_{22} \mathbf{sr}_2 - 2 \cdot \mathbf{sr}_1^\top \mathbf{K}_{12} \mathbf{sr}_2 \quad (70)$$

$$= \text{sMMD}_{k_{TTR}}^{w_{TTR}}(\mathcal{C}^{\mathcal{M}_1}, \mathcal{C}^{\mathcal{M}_2}) \quad (71)$$

where

$$w_{TTR}(\mathcal{C}^{\mathcal{M}_i}) = \mathbf{sr}_i \quad k_{TTR}(\mathcal{C}^{\mathcal{M}_i}[m], \mathcal{C}^{\mathcal{M}_j}[n]) = \mathbf{K}_{ij}[m, n]$$

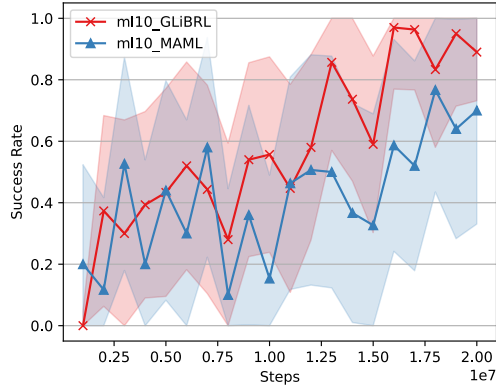
A.10 Implementation Details

We report the details of our implementations of GLiBRL and other methods. The implementation of GLiBRL is simple with only model networks ϕ_T, ϕ_R and policy networks π_ψ^+ . The model networks ϕ_T, ϕ_R are Multi-Layer Perceptrons (MLPs) consisting of *feature* and *mixture* networks. Feature networks convert raw states and actions to features and are shared in ϕ_T and ϕ_R . Mixture networks mix the state and action features, further improving the representativeness. The policy network is actor-only for PPO with linear baseline as suggested by [23] in MetaWorld ML10/45, and actor-critic for SAC in MuJoCo tasks.

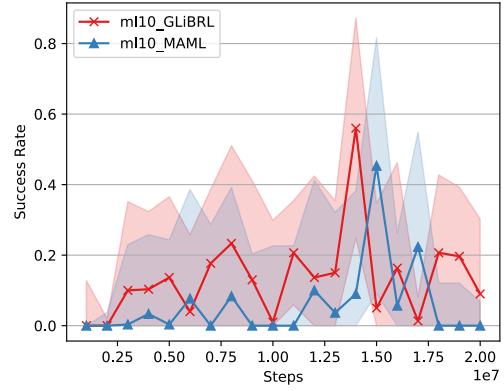
For reproducing all MuJoCo results, we follow the official implementations in TrMRL, PEARL and VariBAD. All results are run with GYMNASIUM==1.2.3 and MUJOCO==3.6.0. For MetaWorld results, we rerun MAML and RL² using the more performant implementation provided by [1], while re-implement and rerun VariBAD as the official implementation does not contain MetaWorld environments. We use the tuned hyperparameters from official implementations in MAML, RL² and VariBAD. For remaining baselines, we use the reported results from [30].

The table of all related hyperparameters of GLiBRL is shown in Appendix A.18.

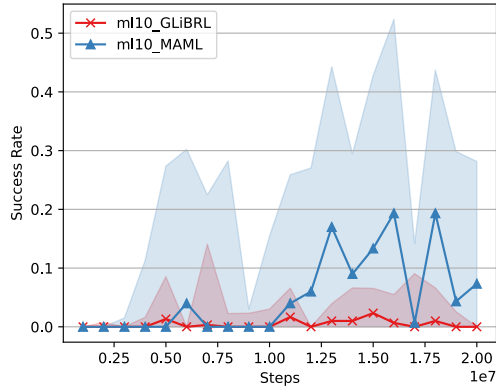
A.11 ML10 Success Rate Comparisons Per-task



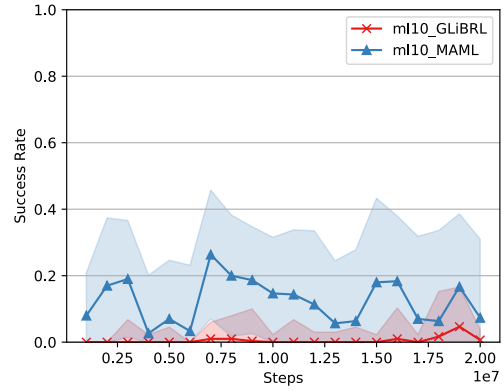
(a) Door Close



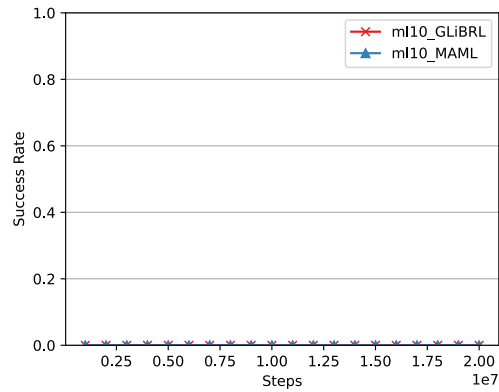
(b) Drawer Open



(c) Lever Pull



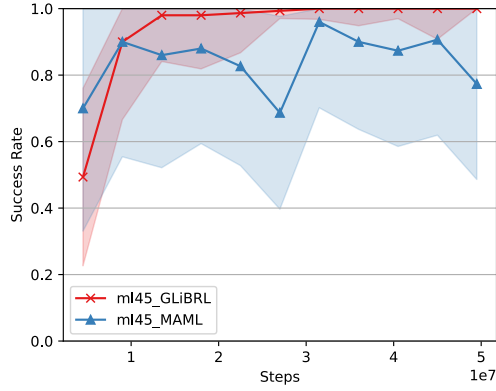
(d) Sweep Into



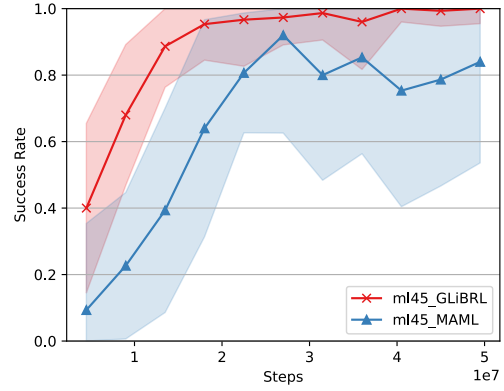
(e) Shelf Place

Figure 2: Testing success rate for each scenario in ML10.

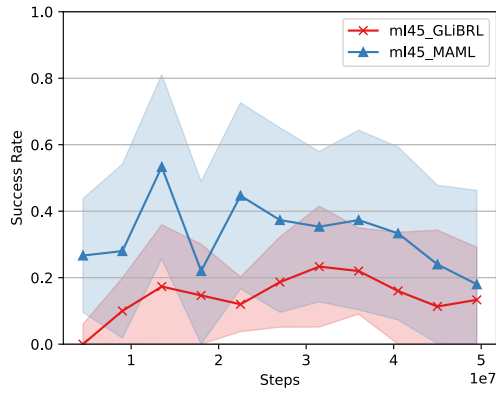
A.12 ML45 Success Rate Comparisons Per-task



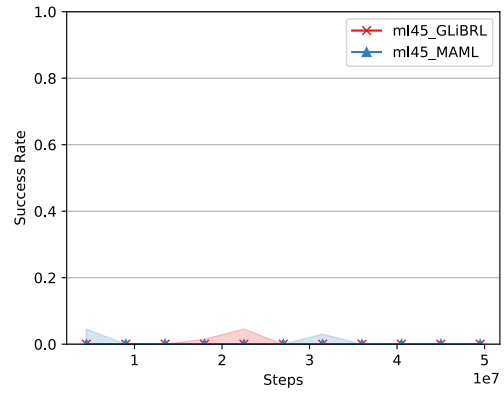
(a) Door Lock



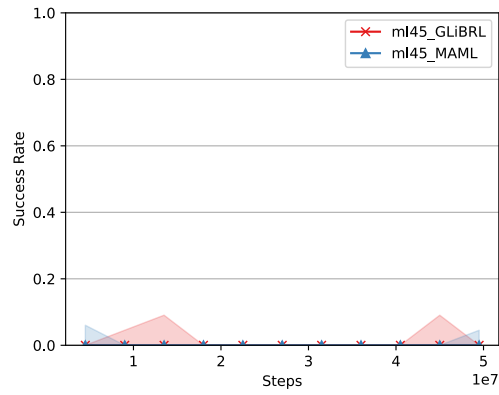
(b) Door Unlock



(c) Hand Insert



(d) Bin Picking



(e) Box Close

Figure 3: Testing success rate for each scenario in ML45. GLiBRL achieves nearly 100% testing success rates in both Door Lock and Door Unlock scenarios.

A.13 Qualitative Results on Task Identifiability

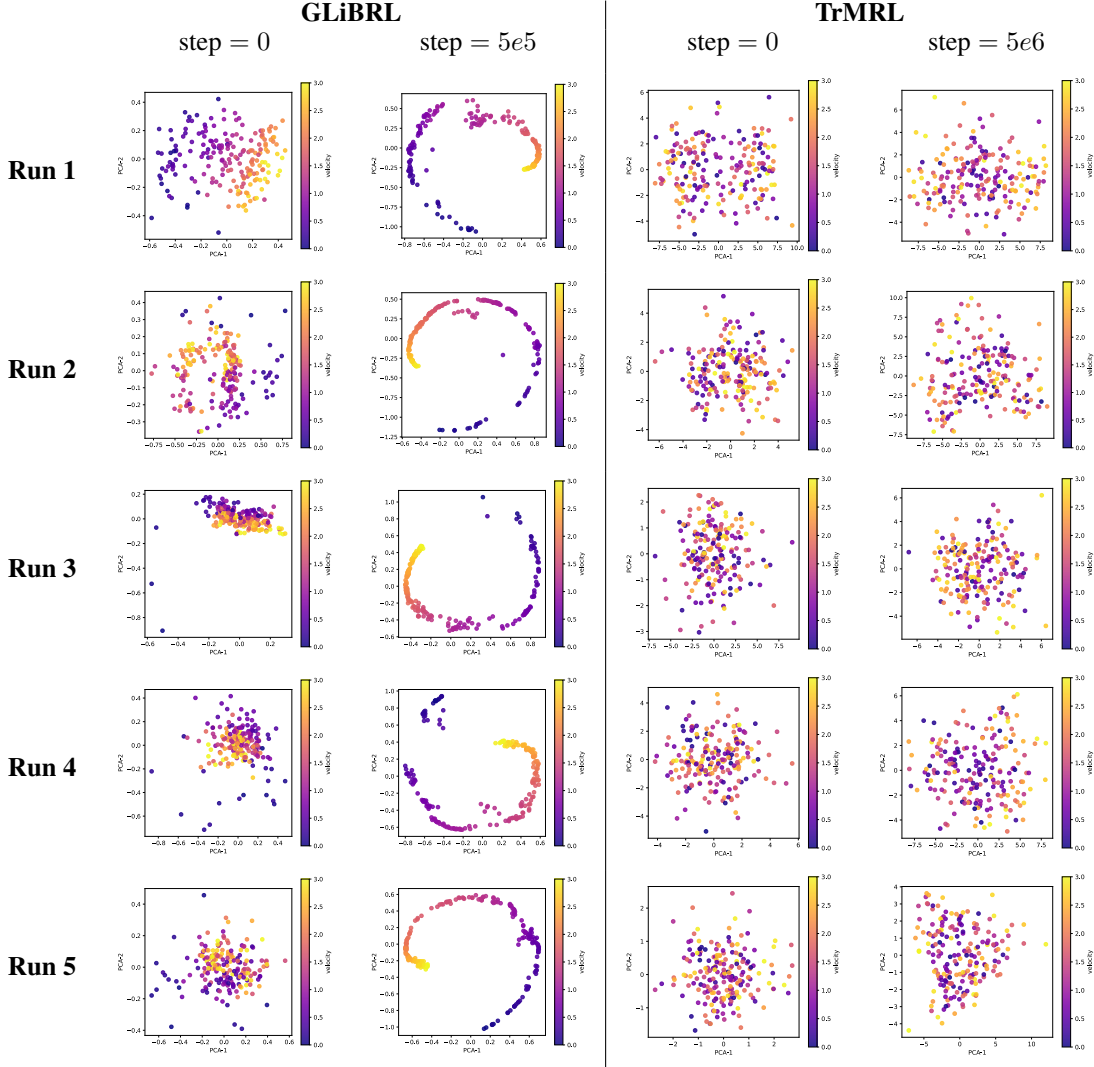


Figure 4: PCA projected task representations of GLiBRL and TrMRL for five independent runs. GLiBRL demonstrates its efficiency and accuracy in distinguishing tasks.

We visualise the task representations of GLiBRL and TrMRL on the HalfCheetahV1 benchmark, projected via 2D PCA across five independent runs. To ensure a rigorous baseline, TrMRL is allocated a $10\times$ larger training step budget. Notably, even prior to any training ($\text{step} = 0$), GLiBRL exhibits better task identifiability than fully trained TrMRL at $5e6$ steps due to its exact Bayesian inference. After merely $5e5$ training steps, GLiBRL perfectly disentangles the task representations, accurately mapping the continuous target velocities into a distinct and highly structured latent space. The spherical pattern matches exactly the normalised representation function $f_{T,R}$. Under GLiBRL, tasks with similar target velocities are similar in the latent space, while tasks with dissimilar target velocities are dissimilar in the latent space.

A.14 Posterior Collapse in VariBAD

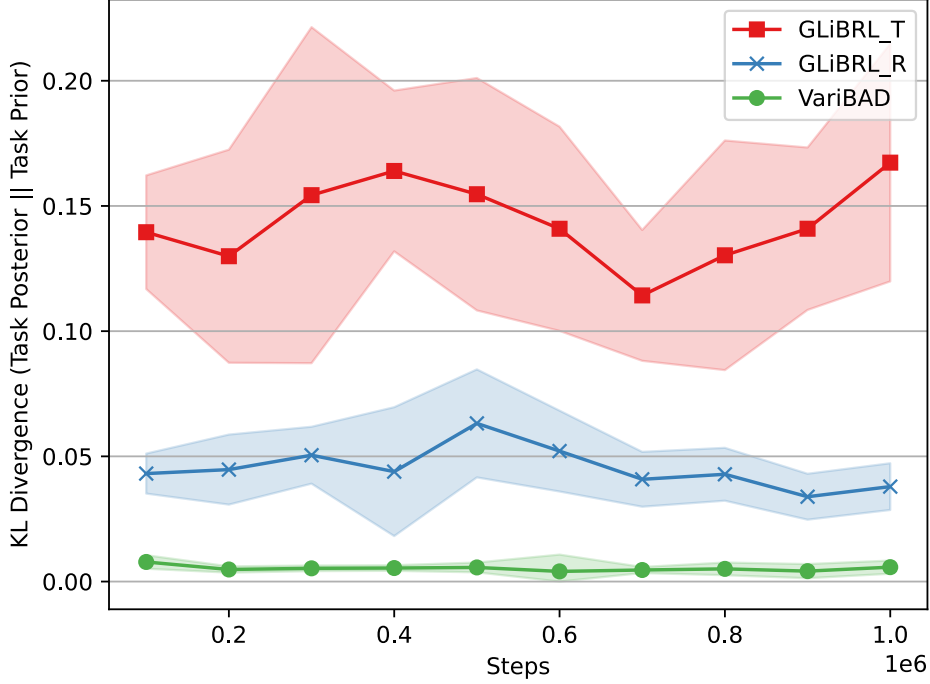


Figure 5: Posterior update KL-divergence for GLiBRL and VariBAD. GLiBRL demonstrates meaningful posterior updates.

To verify if VariBAD can learn meaningful task representations, we check the expected KL-divergence between task posteriors and task priors $\mathbb{E}[D_{KL}(q(\theta_T, \theta_R | \mathcal{C}_{t+1}) || q(\theta_T, \theta_R | \mathcal{C}_t))]$, out of 10 runs in the MetaWorld ML10 benchmark [41, 23], where $\mathcal{C}_{t+1} = \mathcal{C}_t \cup c_{t+1}$ updates the set of samples $\mathcal{C}_t$ with the sample c_{t+1} at time step $t + 1$. VariBAD uses a single latent variable to model both transitions and rewards, hence contributing to only one line in the above figure.

Intuitively, if the expected divergence is close to 0, the majority of posterior updates have collapsed to priors, meaning barely any meaningful task representation has been learnt. Clearly from the above figure, VariBAD shows lower posterior update magnitude and fails to learn meaningful representations, while GLiBRL demonstrates obvious divergence between posteriors and priors.

A.15 Sensitivity of Latent Task Dimensions

To demonstrate the sensitivity of GLiBRL with respect to latent task dimensions D_T and D_R , we perform hyperparameter search in ML10 on (1) $D_T = \{4, 8, 16, 32\}$ while fixing $D_R = 256$ and (2) $D_R = \{32, 64, 128, 256, 512\}$ while fixing $D_T = 16$. We compare on (1) the success rate, (2) the predictive error in transitions, and (3) the predictive error in rewards.

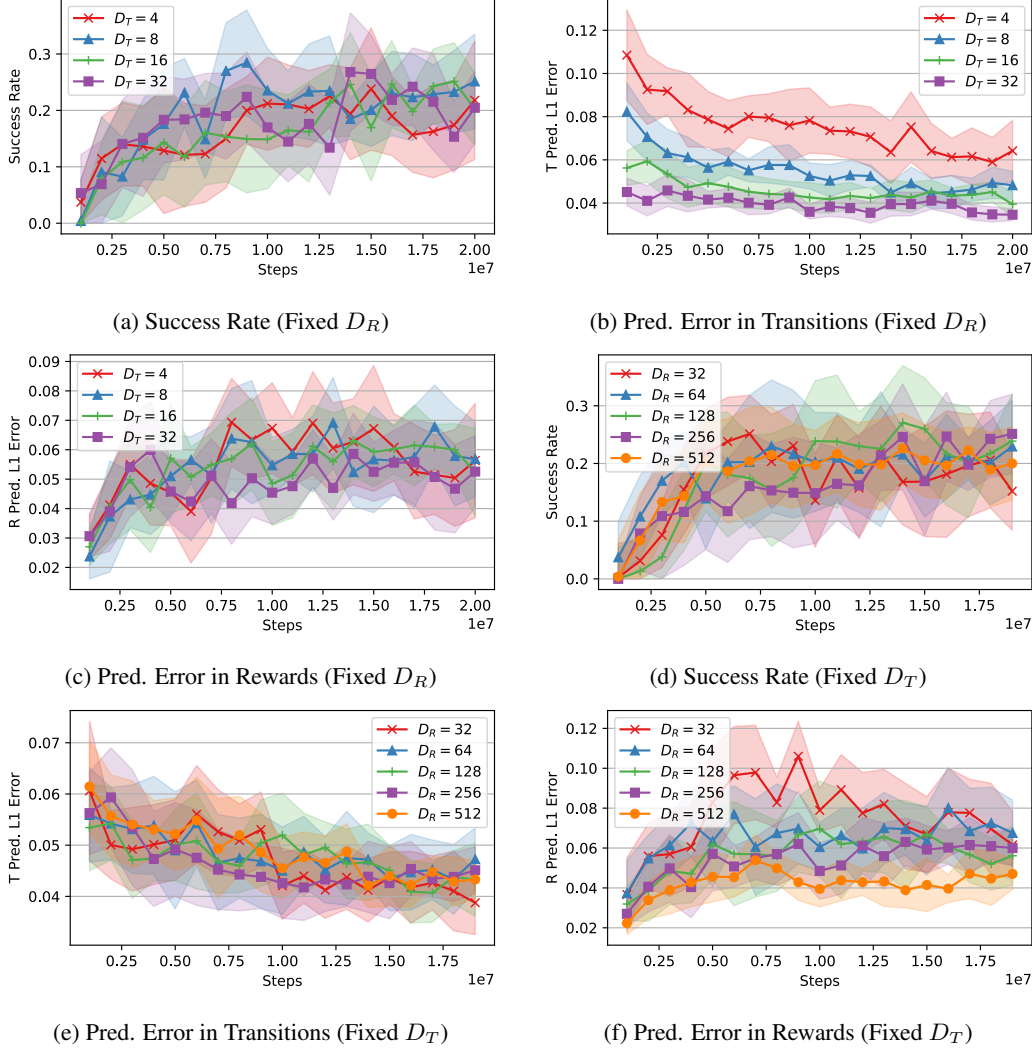


Figure 6: Sensitivity analysis of latent task dimensions in GLiBRL.

In general, GLiBRL shows robust success rates with respect to latent task dimensions D_T , D_R , which can be inferred from Figure (a) and (d). We have observed that GLiBRL achieves state-of-the-art performance on ML10 (29% success rate) with $D_T = 8$, with the cost of predictive errors. Figure (b) and (f) indicate that as D_T , D_R grows, the corresponding error reduces, offering an efficiency-accuracy trade-off. Figure (c) and (e) are sanity checks that have confirmed transitions do not affect rewards, and vice versa. Overall, D_R are set to be much larger than D_T , as reward functions are generally much harder to learn, compared to transition function.

A.16 Ablation Studies

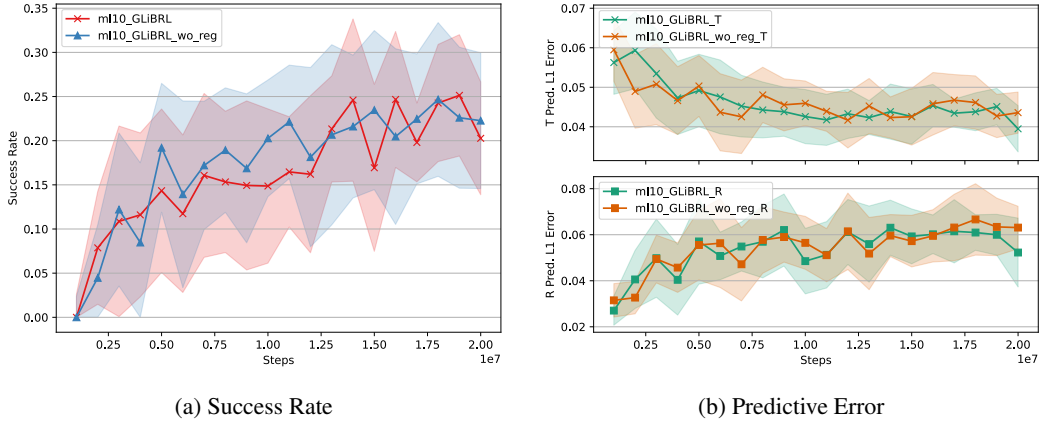


Figure 7: Success rates and predictive errors of GLiBRL with and without regularisation in ML10.

Running GLiBRL and its variant without regularisation in the ML10 benchmark, we can tell the regularisations do not change the overall trend in both success rate and prediction error. However, as the number of training step increases, GLiBRL begins to show in (a): higher success rates with narrower CI; in (b): lower prediction error in both transitions and rewards.

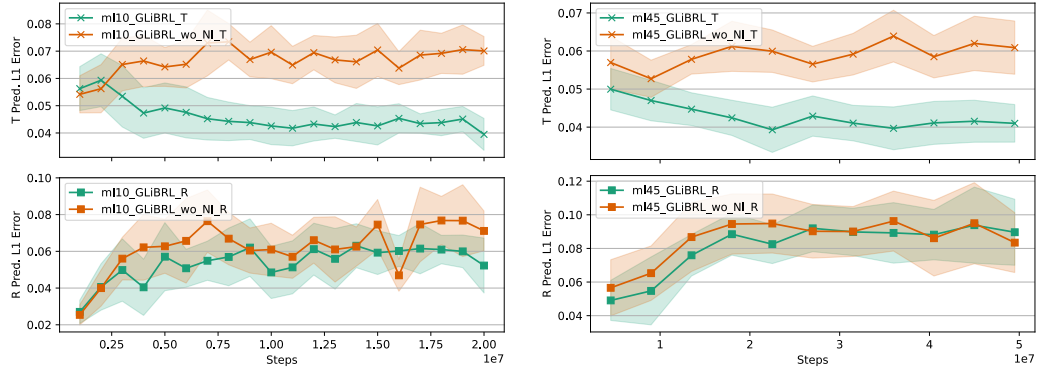


Figure 8: Errors in transition and reward predictions, comparing GLiBRL and GLiBRL_wo_NI. Up: Transitions; Bottom: Rewards; Left: ML10; Right: ML45.

GLiBRL can be viewed as a generalised, deep BRL version of ALPaCA; whereas ALPaCA assumes the model noises $T_\sigma = \Sigma_T, R_\sigma = \Sigma_R$ are fully known a priori, GLiBRL performs full Bayesian inference over them. Under the assumption of ALPaCA, Equation 19 reduces to (see Appendix A.17)

$$\log_{\phi_T, \phi_R}(\mathcal{C}) = -\frac{1}{2} \sum_{i=1}^M \left(D_S \log |\Xi'_T| - \text{Tr}(\Sigma_T^{-1} \mathbf{M}_T'^T \Xi'_T \mathbf{M}_T') \right. \\ \left. + \log |\Xi'_R| - \text{Tr}(\Sigma_R^{-1} \mathbf{M}_R'^T \Xi'_R \mathbf{M}_R') \right) + \text{const.} \quad (72)$$

We studied whether performing Bayesian inference on model noises is necessary for learning accurate transition and reward models. GLiBRL and its variant without noise inference (GLiBRL_wo_NI) are tested with identical hyperparameters on both ML10 and ML45 benchmarks⁹. The metrics being evaluated are $\mathcal{L}_1$ norms of prediction errors in both transitions (defined as $|\mathbf{S}' - \mathbf{C}_T \mathbf{M}_T'|_1$) and rewards (defined as $|\mathbf{r} - \mathbf{C}_R \mathbf{M}_R'|_1$).¹⁰

⁹They also have the same initial noises. $(\nu_T \mathbf{\Omega}_T)^{-1} = \Sigma_T = 0.025 \cdot \mathbf{I}$ and $(\nu_R \mathbf{\Omega}_R)^{-1} = \Sigma_R = 0.5$.

¹⁰Comparisons of success rates are not included, as there is no obvious difference.

As demonstrated in Figure 8, GLiBRL achieves lower prediction errors across both transitions and rewards compared to the GLiBRL_wo_NI which assumes known noises. Both methods become increasingly erroneous in reward predictions with training steps. This is expected, as more steps result in higher success rates, hence increased magnitude of rewards and errors.

However, the increasing trend of transition errors of GLiBRL_wo_NI is pathological, as magnitudes of states remain relatively bounded and less relevant with the success rate. The lower prediction error of GLiBRL allows better integrations with model-based methods using imaginary samples, whose quality depends highly on the accuracy of the prediction.

A.17 Marginal Log-likelihood of Normal-Normal

We prove Equation 72 has the following form

$$\begin{aligned} \log_{\phi_T, \phi_R}(\mathcal{C}) = & -\frac{1}{2} \sum_{i=1}^M \left(D_S \log |\Xi'_T| - \text{Tr}(\Sigma_T^{-1} \mathbf{M}'_T \Xi'_T \mathbf{M}'_T) \right. \\ & \left. + \log |\Xi'_R| - \text{Tr}(\Sigma_R^{-1} \mathbf{M}'_R \Xi'_R \mathbf{M}'_R) \right) + \text{const.} \end{aligned} \quad (72)$$

Proof:

The distributions without inferring on the noise are listed as follows:

Likelihood:

$$p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i, \theta_T) = \mathcal{MN}(\mathbf{S}'_i | \mathbf{C}_T T_\mu, \mathbf{I}_N, \Sigma_T \in \mathbb{R}^{D_S \times D_S}) \quad (73)$$

$$p_{\phi_R}(\mathbf{r}_i | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i, \theta_R) = \mathcal{MN}(\mathbf{r}_i | \mathbf{C}_R R_\mu, \mathbf{I}_N, \Sigma_R \in \mathbb{R}^{1 \times 1}) \quad (74)$$

Prior:

$$p(\theta_T) = p(T_\mu) = \mathcal{MN}(T_\mu | \mathbf{M}_T \in \mathbb{R}^{D_T \times D_S}, \Xi_T^{-1} \in \mathbb{R}^{D_T \times D_T}, \Sigma_T) \quad (75)$$

$$p(\theta_R) = p(R_\mu) = \mathcal{MN}(R_\mu | \mathbf{M}_R \in \mathbb{R}^{D_R \times 1}, \Xi_R^{-1} \in \mathbb{R}^{D_R \times D_R}, \Sigma_R) \quad (76)$$

Posterior:

$$p_{\phi_T}(\theta_T | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i) = \mathcal{MN}(T_\mu | \mathbf{M}'_T, \Xi_T'^{-1}, \Sigma_T) \quad (77)$$

$$p_{\phi_R}(\theta_R | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i, \mathbf{r}) = \mathcal{MN}(R_\mu | \mathbf{M}'_R, \Xi_R'^{-1}, \Sigma_R) \quad (78)$$

where

$$\begin{aligned} \mathbf{M}'_T &= \Xi_T'^{-1} [\mathbf{C}_T^T \mathbf{S}' + \Xi_T \mathbf{M}_T] & \mathbf{M}'_R &= \Xi_R'^{-1} [\mathbf{C}_R^T \mathbf{r} + \Xi_R \mathbf{M}_R] \\ \Xi'_T &= \mathbf{C}_T^T \mathbf{C}_T + \Xi_T & \Xi'_R &= \mathbf{C}_R^T \mathbf{C}_R + \Xi_R \end{aligned} \quad (79)$$

Similar to Appendix A.7,

$$p_{\phi_T}(\mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i) = \frac{p_{\phi_T}(\theta_T, \mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i)}{p_{\phi_T}(\theta_T | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i)} \quad (80)$$

As

$$\begin{aligned} p_{\phi_T}(\theta_T, \mathbf{S}'_i | \mathbf{S}_i, \mathbf{A}_i) &= \frac{|\Xi_T'|^{D_S/2} |\Sigma_T|^{-D_T/2}}{\sqrt{(2\pi)^{D_T D_S}}} \cdot \exp \left[-\frac{1}{2} \text{Tr}(\Sigma_T^{-1} (T_\mu - \mathbf{M}_T)^T \Xi_T (T_\mu - \mathbf{M}_T)) \right] \\ &\quad \cdot \frac{|\Sigma_T|^{-N/2}}{\sqrt{(2\pi)^{N D_S}}} \cdot \exp \left[-\frac{1}{2} \text{Tr}(\Sigma_T^{-1} (\mathbf{S}'_i - \mathbf{C}_T T_\mu)^T (\mathbf{S}'_i - \mathbf{C}_T T_\mu)) \right] \end{aligned} \quad (81)$$

And

$$p_{\phi_T}(\theta_T | \mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i) = \frac{|\Xi_T'|^{D_S/2} |\Sigma_T|^{-D_T/2}}{\sqrt{(2\pi)^{D_T D_S}}} \cdot \exp \left[-\frac{1}{2} \text{Tr}(\Sigma_T^{-1} (T_\mu - \mathbf{M}'_T)^T \Xi'_T (T_\mu - \mathbf{M}'_T)) \right] \quad (82)$$

Hence

$$p_{\phi_T}(\mathbf{S}'_i|\mathbf{S}_i, \mathbf{A}_i) \propto |\boldsymbol{\Xi}'_T|^{-D_S/2} \cdot \exp \left[-\frac{1}{2} \text{Tr} \left(-\boldsymbol{\Sigma}_T^{-1} \mathbf{M}_T^T \boldsymbol{\Xi}'_T \mathbf{M}'_T \right) \right] \quad (83)$$

$$\log p_{\phi_T}(\mathbf{S}'_i|\mathbf{S}_i, \mathbf{A}_i) = -\frac{D_S}{2} \log |\boldsymbol{\Xi}'_T| + \frac{1}{2} \text{Tr} \left(\boldsymbol{\Sigma}_T^{-1} \mathbf{M}_T^T \boldsymbol{\Xi}'_T \mathbf{M}'_T \right) + \text{const.} \quad (84)$$

Similarly,

$$\log p_{\phi_R}(\mathbf{r}_i|\mathbf{S}_i, \mathbf{A}_i, \mathbf{S}'_i) = -\frac{1}{2} \log |\boldsymbol{\Xi}'_R| + \frac{1}{2} \text{Tr} \left(\boldsymbol{\Sigma}_R^{-1} \mathbf{M}_R^T \boldsymbol{\Xi}'_R \mathbf{M}'_R \right) + \text{const.} \quad (85)$$

Following Appendix A.7,

$$\begin{aligned} \log_{\phi_T, \phi_R}(\mathcal{C}) = & -\frac{1}{2} \sum_{i=1}^M \left(D_S \log |\boldsymbol{\Xi}'_T| - \text{Tr} \left(\boldsymbol{\Sigma}_T^{-1} \mathbf{M}_T^T \boldsymbol{\Xi}'_T \mathbf{M}'_T \right) \right. \\ & \left. + \log |\boldsymbol{\Xi}'_R| - \text{Tr} \left(\boldsymbol{\Sigma}_R^{-1} \mathbf{M}_R^T \boldsymbol{\Xi}'_R \mathbf{M}'_R \right) \right) + \text{const.} \end{aligned} \quad (72)$$

A.18 Hyperparameters, Runtime and Memory

We list all hyperparameters of GLiBRL in the following tables.

In HalfCheetahDir:

Name	Value	Name	Value
policy_learner	SAC	feat_out_activation	True
critic_layer	[256, 256]	t_mix_layers	[64, 32]
actor_layer	[256, 256, 256]	t_mix_layernorm	True
policy_activation	Tanh	t_mix_out_activation	False
policy_optimiser	Adam	r_mix_layers	[128, 64]
actor_lr/critic_lr	3e-4/4e-4	r_mix_layernorm	True
policy_max_norm	10	r_mix_out_activation	False
policy_weight_init	Xavier	t_reg_coef	5e-3
policy_bias_init	0	r_reg_coef	1e-3
actor_logstd_range	$[\ln 10^{-6}, \ln 2]$	model_activation	ReLU
tau	5e-3	model_optimiser	Adam
actor_grad_steps	500	model_lr	2e-4
critic_grad_steps	500	model_opt_max_norm	None
policy_update_freq	200	model_grad_epochs	1
init_alpha	1	model_grad_steps	1
alpha_lr	3e-4	init_mt	zeros
policy_bs_range	[1, 200]	init_mr	zeros
s_feat_layers	[64, 32]	init_xit	ones
s_feat_outdim	32	init_xir	ones
s_feat_layernorm	False	init_omegat	ones
a_feat_layers	[32, 16]	init_omegar	ones
a_feat_outdim	16	init_nut	22
a_feat_layernorm	False	init_nur	2

In HalfCheetahVel:

Name	Value	Name	Value
policy_learner	SAC	feat_out_activation	True
policy_layers	[256, 256]	t_mix_layers	[64, 32]
a_feat_out_activation	True	t_mix_layernorm	True
policy_activation	Tanh	t_mix_out_activation	False
policy_optimiser	Adam	r_mix_layers	[128, 64]
actor_lr/critic_lr	3e-4/4e-4	r_mix_layernorm	True
policy_max_norm	10	r_mix_out_activation	False
policy_weight_init	Xavier	t_reg_coef	5e-3
policy_bias_init	0	r_reg_coef	1e-3
actor_logstd_range	$[\ln 10^{-6}, \ln 2]$	model_activation	ReLU
tau	1e-2	model_optimiser	Adam
actor_grad_steps	2000	model_lr	2e-4
critic_grad_steps	2000	model_opt_max_norm	None
policy_update_freq	200	model_grad_epochs	1
init_alpha	0.2	model_grad_steps	50
alpha_lr	3e-4	init_mt	zeros
policy_bs_range	[1, 200]	init_mr	zeros
s_feat_layers	[64, 32]	init_xit	ones
s_feat_outdim	32	init_xir	ones
s_feat_layernorm	False	init_omegat	ones
a_feat_layers	[32, 16]	init_omegar	ones
a_feat_outdim	16	init_nut	22
a_feat_layernorm	False	init_nur	2

In AntDir:

Name	Value	Name	Value
policy_learner	SAC	feat_out_activation	True
policy_layers	[256, 256]	t_mix_layers	[64, 32]
a_feat_out_activation	True	t_mix_layernorm	True
policy_activation	Tanh	t_mix_out_activation	False
policy_optimiser	Adam	r_mix_layers	[128, 64]
actor_lr/critic_lr	3e-4/4e-4	r_mix_layernorm	True
policy_max_norm	1	r_mix_out_activation	False
policy_weight_init	Xavier	t_reg_coef	5e-3
policy_bias_init	0	r_reg_coef	1e-3
actor_logstd_range	$[\ln 10^{-6}, \ln 2]$	model_activation	ReLU
tau	1e-2	model_optimiser	Adam
actor_grad_steps	1000	model_lr	2e-4
critic_grad_steps	2000	model_opt_max_norm	None
policy_update_freq	400	model_grad_epochs	1
init_alpha	0.1	model_grad_steps	5
alpha_lr	3e-4	init_mt	zeros
policy_bs_range	[1, 200]	init_mr	zeros
s_feat_layers	[64, 32]	init_xit	ones
s_feat_outdim	32	init_xir	ones
s_feat_layernorm	False	init_omegat	ones
a_feat_layers	[32, 16]	init_omegar	ones
a_feat_outdim	16	init_nut	28
a_feat_layernorm	False	init_nur	2

We use the same hyperparameters for both ML10 and ML45.

Name	Value	Name	Value
policy_learner	PPO	feat_out_activation	True
policy_layers	[256, 256]	t_mix_layers	[64, 32]
a_feat_out_activation	True	t_mix_layernorm	True
policy_activation	Tanh	t_mix_out_activation	False
policy_optimiser	Adam	r_mix_layers	[128, 64]
policy_lr	5e-4	r_mix_layernorm	True
policy_opt_max_norm	N/A	r_mix_out_activation	False
policy_weight_init	Xavier	t_reg_coef	5e-3
policy_bias_init	0	r_reg_coef	1e-3
policy_log_std_min	$\ln 10^{-6}$	model_activation	ReLU
policy_log_std_max	$\ln 2$	model_optimiser	Adam
policy_grad_epochs	10	model_lr	2e-4
policy_grad_steps	20	model_opt_max_norm	None
ppo_clip_eps	0.5	model_grad_epochs	1
ppo_gamma	0.99	model_grad_steps	20
ppo_gae_lambda	0.95	init_mt	zeros
ppo_entropy_coef	5e-3	init_mr	zeros
s_feat_layers	[64, 32]	init_xit	ones
s_feat_outdim	32	init_xir	ones
s_feat_layernorm	False	init_omegat	ones
a_feat_layers	[32, 16]	init_omegar	ones
a_feat_outdim	16	init_nut	40
a_feat_layernorm	False	init_nur	2

GLiBRL is rather efficient in both time and memory. Although all of our experiments are run using A100, we have tested that GLiBRL can run fast on much lower-end GPUs with 8GB memory, such as RTX 3070, with each run costing less than 3 hours. The runtime does not vary much with changes in D_T and D_R , despite the quadratic online inference complexity.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The claims and the results are consistent.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: In the Conclusion section, we mentioned the limitations.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[Yes\]](#)

Justification: All proofs are in the appendix. We state assumptions whenever needed.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: In the appendix we mentioned the implementation details and hyperparameters.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Experiments are reproducible from the provided supplementary zip file.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: We follow the exact procedure from previous papers, and provided hyperparameters in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We run experiments five times for MuJoCo and ten times for MetaWorld. We reported the 95% CI.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).

- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: In both experiment section and appendix we mentioned the resource.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We checked the Code of Ethics and do not observe any violations.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: The goal of this paper is to forward the research of Meta-RL / Bayes-RL. The authors do not see a potential societal impact of this work.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper poses no such risks. We use open source datasets without any modifications.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We use open source datasets and cited them.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: We use existing datasets.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: All experiments are performed in simulation. No human participants involved.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: All experiments are performed in simulation. No human participants involved.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLM is only used for improving the writing.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.